

UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA

Dipartimento di Informatica, Sistemistica e Comunicazione (DISCo)

Corso di laurea in Data Science

Studio di Robustezza dei Modelli di Machine Learning al Rumore nei Dati

Progetto di *Architetture Dati*

Un'analisi sperimentale sul dataset MAGIC Gamma Telescope

Autori

Daniele Maccagnan – matricola [matricola]

Matias Bonoli – matricola [matricola]

Ashley Chudory – matricola [matricola]

Anno accademico 2025/2026

Indice

1	Introduzione	2
2	Descrizione del Dataset	2
3	Suddivisione in Training, Validation e Test	3
4	Analisi Esplorativa dei Dati	3
5	Preprocessing: Standardizzazione	7
6	Modelli	8
7	Baseline su Dati Puliti	9
8	Metodologia di Iniezione del Rumore	11
8.1	Protocollo comune e livelli di intensità	11
8.2	Descrizione dei sei meccanismi	12
9	Risultati per Tipo di Rumore	13
9.1	Feature Noise	13
9.2	Label Noise Random	16
9.3	Label Noise Borderline	19
9.4	Missing Values	22
9.5	Outlier	24
9.6	Missing Feature (sensore guasto)	26
10	Sintesi Trasversale	29
11	Analisi Non Supervisionata (K-Means)	32
12	Conclusioni	33

1 Introduzione

La qualità dei dati è raramente perfetta: misurazioni imprecise, errori di annotazione, valori mancanti e anomalie sono la norma in qualunque pipeline reale. Per questo, prima ancora di chiederci quale modello raggiunga l'accuratezza più alta su dati puliti, riteniamo più utile chiederci quanto ciascun modello *resista* quando i dati si degradano. Questo progetto affronta esattamente tale domanda: misuriamo in modo sistematico la robustezza di tre modelli di apprendimento supervisionato quando il dataset di addestramento viene progressivamente “sporcolato” con sei tipi di rumore distinti.

L'obiettivo non è dichiarare un vincitore assoluto, ma capire *come* e *perché* ogni modello si degrada in funzione della natura del disturbo. A questo scopo abbiamo adottato un protocollo uniforme: fissato un riferimento su dati puliti, abbiamo riaddestrato gli stessi modelli su versioni del training set corrotte a tre livelli di intensità crescente (10%, 25% e 40%), confrontando poi le metriche calcolate sempre sullo stesso test set pulito. I tre modelli sono una rete neurale semplice (*NN Base*), una sua versione regolarizzata (*NN Pro*) e un *Random Forest*; i sei rumori coprono disturbi sulle feature, sulle etichette, sui valori mancanti, sugli outlier e sulla disponibilità delle feature stesse. A chiusura, abbiamo affiancato un esperimento non supervisionato (K-Means) per osservare come il rumore agisca quando le etichette non vengono affatto utilizzate.

La struttura della relazione segue passo per passo il flusso del lavoro sperimentale. Dopo aver descritto il dataset, presentiamo la suddivisione dei dati e l'analisi esplorativa condotta sul solo insieme di addestramento; spieghiamo poi il preprocessing, i tre modelli e il loro comportamento sui dati puliti; quindi definiamo nel dettaglio i sei meccanismi di rumore e ne analizziamo gli effetti uno alla volta; infine tiriamo le fila con una sintesi trasversale, l'esperimento non supervisionato e le conclusioni. Tutti i numeri e le figure provengono direttamente dall'esecuzione del notebook del progetto: non abbiamo riaddestrato i modelli né ricalcolato alcuna metrica, così da garantire piena coerenza tra il codice e ciò che descriviamo. Per rendere i risultati riproducibili abbiamo fissato un seme casuale comune (`seed = 42`) a tutte le componenti stocastiche: la suddivisione dei dati, l'inizializzazione delle reti, l'iniezione del rumore, il Random Forest e il K-Means.

2 Descrizione del Dataset

Il dataset utilizzato è il **MAGIC Gamma Telescope**, prodotto da simulazioni Monte Carlo del telescopio Cherenkov MAGIC. Ogni riga descrive la traccia luminosa, di forma all'incirca ellittica, registrata sul piano del rivelatore da un evento atmosferico; il problema consiste nel distinguere i segnali utili, generati da raggi *gamma*, dal fondo di *adroni* (raggi cosmici). Si tratta quindi di un problema di classificazione binaria con un chiaro significato fisico: classificare un adrone come gamma (falso positivo) introduce contaminazione nel segnale, mentre scartare un gamma vero (falso negativo) riduce la statistica utile. Questa asimmetria di costo è la ragione per cui, più avanti, non ci accontenteremo della sola accuratezza.

Il file di partenza contiene 19 020 osservazioni e 11 colonne: dieci feature continue, che riassumono la geometria e la distribuzione di intensità dell'immagine dell'evento, e una variabile target. Abbiamo caricato i dati assegnando i nomi delle colonne e mappato le due classi originali in valori numerici ($h \rightarrow 0$ per gli adroni, $g \rightarrow 1$ per i gamma), così da poterle trattare direttamente come bersaglio di una classificazione binaria. La Tabella 1

riassume il significato delle dieci feature.

Tabella 1: Le dieci feature continue del dataset MAGIC e il loro significato.

Feature	Descrizione
fLength	lunghezza dell'asse maggiore dell'ellisse dell'immagine
fWidth	lunghezza dell'asse minore dell'ellisse
fSize	logaritmo della somma delle intensità di tutti i pixel
fConc	rapporto di concentrazione dei due pixel più intensi
fConc1	rapporto di concentrazione del pixel più intenso
fAsym	asimmetria della distribuzione lungo l'asse maggiore
fM3Long	momento terzo (asimmetria) lungo l'asse maggiore
fM3Trans	momento terzo lungo l'asse minore
fAlpha	angolo tra l'asse maggiore e la direzione del centro del campo
fDist	distanza dal centro del campo all'origine dell'ellisse
class	target: 1 = gamma (segnale), 0 = adrone (fondo)

3 Suddivisione in Training, Validation e Test

Una scelta metodologica che riteniamo importante riguarda l'*ordine* delle operazioni: abbiamo suddiviso il dataset in tre parti *prima* di qualsiasi altra elaborazione, inclusa l'analisi esplorativa. Lavorare fin dall'inizio su insiemi separati evita che osservazioni o statistiche calcolate sull'intero dataset condizionino, anche indirettamente, le decisioni prese in seguito; in particolare, sia l'esplorazione dei dati sia tutte le statistiche di pre-processing useranno il *solo* training set, in modo che validation e test restino una stima onesta della capacità di generalizzazione.

La suddivisione è stata realizzata in due passaggi successivi, ottenendo proporzioni 60% / 20% / 20% rispettivamente per training, validation e test. Entrambi i passaggi sono *stratificati* sulla variabile target, cioè mantengono in ogni sottoinsieme le stesse proporzioni tra gamma e adroni presenti nel dataset completo: è una precauzione necessaria proprio perché le classi sono sbilanciate, e senza stratificazione un sottoinsieme potrebbe per caso contenere troppi pochi adroni. In termini assoluti la ripartizione produce 11 412 campioni di addestramento, 3 804 di validation e 3 804 di test. Il validation set ha un ruolo operativo preciso che useremo più avanti: monitorare l'andamento dell'addestramento delle reti (le curve di train contro validation) e guidare l'arresto anticipato della rete regolarizzata.

4 Analisi Esplorativa dei Dati

L'analisi esplorativa è stata condotta esclusivamente sul training set, coerentemente con la scelta di isolare validation e test descritta nella Sezione 3. Una prima verifica ha mostrato l'assenza completa di valori nulli: la condizione di partenza è quindi "pulita", il che rende il nostro studio particolarmente controllato, perché tutta la sporcizia analizzata sarà introdotta in seguito da noi in modo deliberato e quantificabile. Abbiamo inoltre individuato e rimosso 41 righe duplicate nel training set, portandolo da 11 412 a 11 371 osservazioni: eliminare i duplicati evita che campioni identici pesino più del dovuto in alcune zone dello spazio delle feature.

La Figura 1 mostra la distribuzione del target e conferma a colpo d'occhio uno sbilanciamento marcato: i gamma costituiscono circa il 64,8% delle osservazioni e gli adroni il restante 35,2%, quasi un rapporto di due a uno. Questo squilibrio ha una conseguenza pratica immediata: un classificatore banale che predicesse sempre la classe maggioritaria otterrebbe già un'accuratezza vicina al 65%. Tale valore rappresenta la soglia minima al di sopra della quale un risultato comincia a essere informativo, e sarà il nostro riferimento implicito ogni volta che leggeremo un'accuratezza.

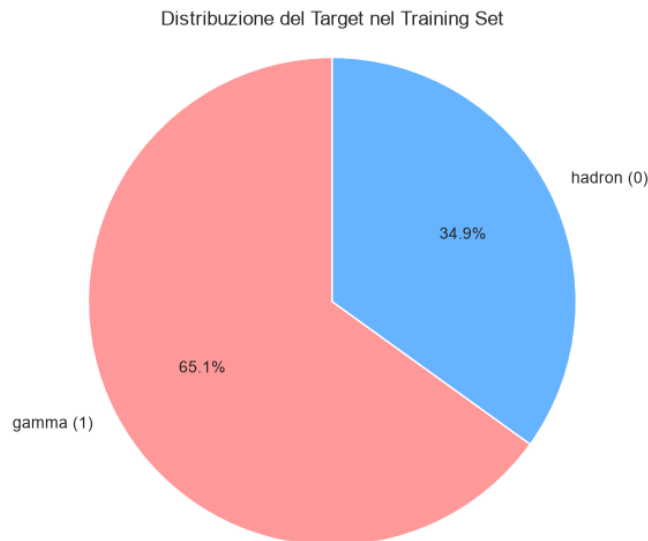


Figura 1: Distribuzione delle due classi nel training set: i gamma (segnale) sono quasi il doppio degli adroni (fondo).

Osservando le distribuzioni delle singole feature (Figura 2) notiamo che quasi tutte sono fortemente asimmetriche, concentrate verso valori bassi e con code lunghe verso destra. L'asimmetria è una proprietà statistica rilevante perché implica che la *mediana*, meno sensibile ai valori estremi, descrive il centro di queste distribuzioni in modo più rappresentativo della media. Notiamo inoltre che le feature vivono su intervalli numerici molto diversi tra loro: alcune si misurano in poche unità, altre in centinaia, una differenza di scala di cui terremo conto nel preprocessing.

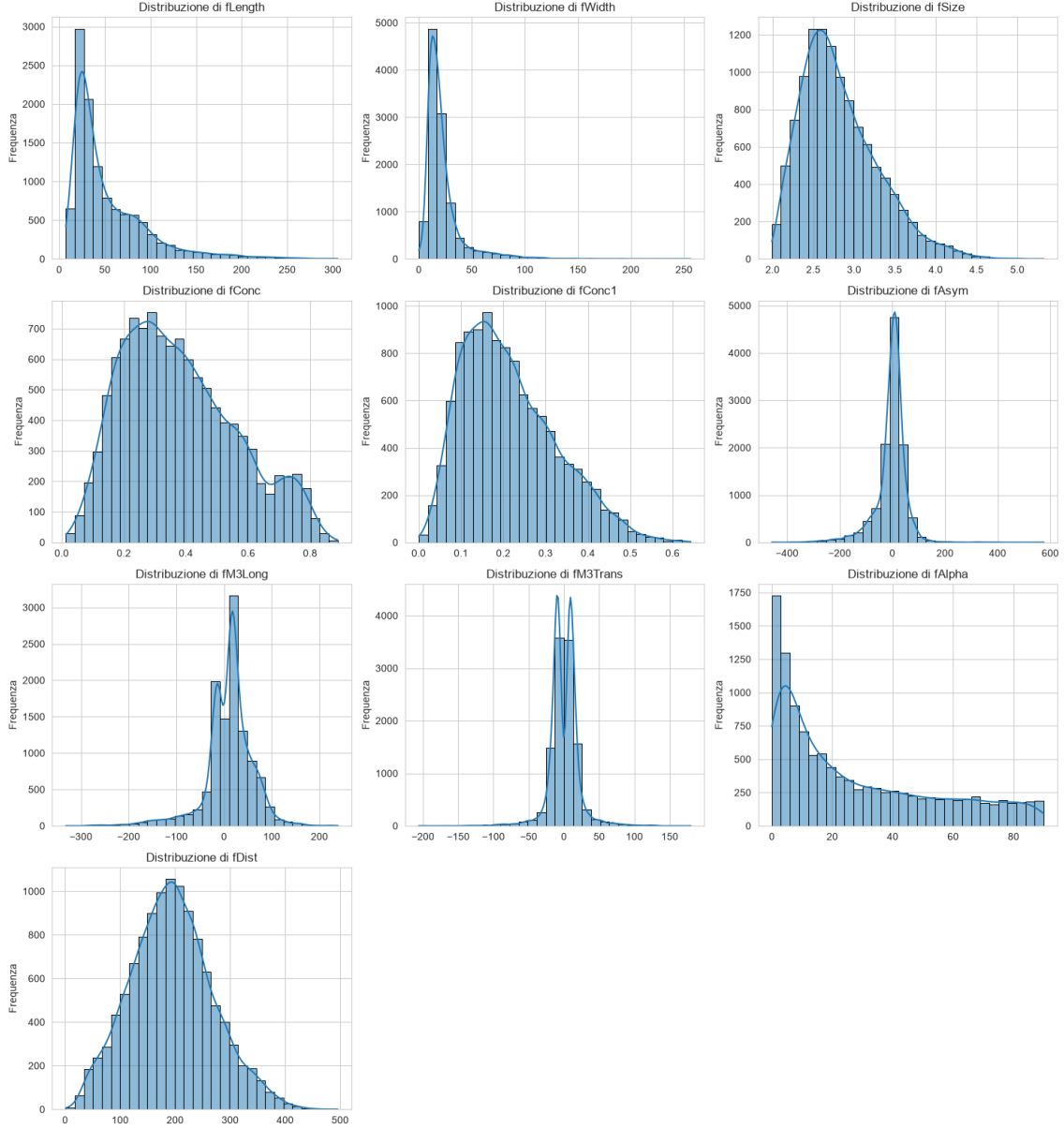


Figura 2: Istogrammi delle dieci feature continue (training set). La maggior parte presenta distribuzioni asimmetriche con code pronunciate e scale numeriche eterogenee.

I boxplot per classe (Figura 3) sono particolarmente istruttivi perché mostrano *quali* feature separano davvero gamma e adroni. La feature **fAlpha** presenta la differenza più netta tra le due classi, con distribuzioni nettamente diverse: per gli adroni assume valori distribuiti su un ampio intervallo, mentre per i gamma si concentra su valori piccoli. La seguono **fLength**, **fWidth** e **fSize**, anch'esse con un buon potere discriminante, mentre alcune feature, come **fM3Trans** e **fAsym**, hanno distribuzioni quasi sovrapposte tra le due classi e contribuiscono poco alla separazione. Già da questa osservazione emerge che l'informazione utile non è distribuita uniformemente tra le feature, ma è concentrata in poche di esse.

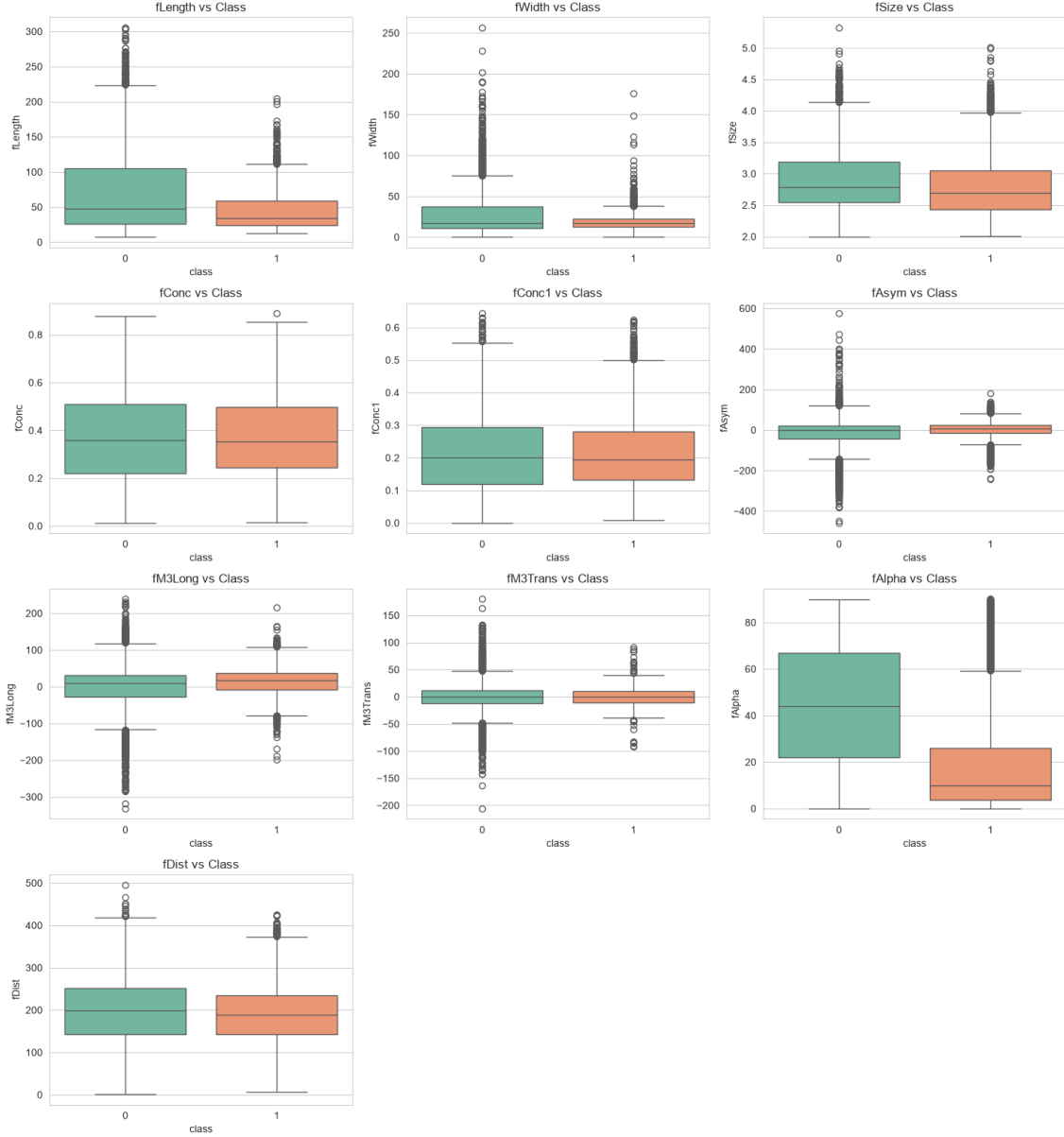


Figura 3: Boxplot di ogni feature separato per classe (training set). **fAlpha** è la più discriminante; le feature dei momenti trasversali separano poco le classi.

Infine, la matrice di correlazione (Figura 4) rivela alcune relazioni lineari forti. Le due feature di concentrazione **fConc** e **fConc1** sono quasi ridondanti (correlazione 0,98), e **fSize** è fortemente anticorrelata con entrambe ($-0,85$ e $-0,81$); anche la terzina **fLength**, **fWidth**, **fSize** mostra correlazioni elevate (0,71–0,77). Questa ridondanza significa che diverse feature trasportano informazione in parte sovrapposta, e che quindi la perdita o il disturbo di una di esse può essere in parte compensato dalle altre. All'estremo opposto osserviamo che **fAlpha**, pur essendo la più discriminante, è quasi scorrelata da tutte le altre: la sua informazione è quindi sostanzialmente unica e non recuperabile altrove. Terremo presente entrambe queste osservazioni quando interpreteremo gli effetti del rumore.

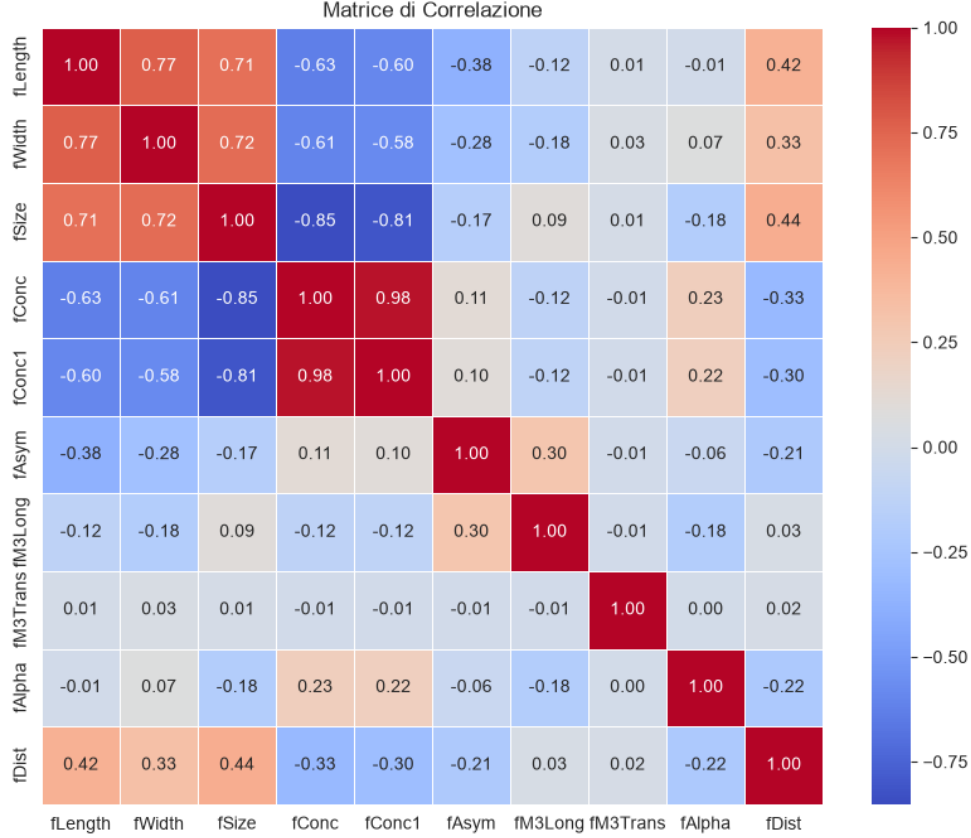


Figura 4: Matrice di correlazione tra le feature (training set). Spiccano la quasi-ridondanza `fConc/fConc1` e l’anticorrelazione di `fSize` con le concentrazioni; `fAlpha` è quasi scorrelata da tutte.

5 Preprocessing: Standardizzazione

Alla luce delle scale numeriche eterogenee osservate nell’analisi esplorativa (Sezione 4), abbiamo standardizzato le feature portandole a media nulla e deviazione standard unitaria. Questo passaggio è importante soprattutto per le reti neurali, la cui convergenza beneficia di input su scale omogenee: senza standardizzazione, le feature dai valori più grandi dominerebbero il calcolo dei gradienti semplicemente per via della loro scala, e non per la loro effettiva rilevanza.

Coerentemente con il principio enunciato nella Sezione 3, lo standardizzatore è stato calcolato *soltanto* sul training set e poi applicato, con gli stessi parametri, a validation e test. In questo modo nessuna informazione sui dati di validazione o di test “trapela” nel preprocessing (assenza di *data leakage*): media e deviazione standard usate per la trasformazione provengono unicamente dai dati di addestramento. La standardizzazione ha anche un effetto collaterale utile per il seguito, perché porta tutte le feature su una scala comune in cui “una deviazione standard” significa la stessa cosa per ognuna; ciò renderà interpretabili e confrontabili i livelli di rumore che introdurremo.

6 Modelli

Abbiamo confrontato tre modelli scelti per rappresentare livelli crescenti di difesa contro il rumore. La Tabella 2 ne riassume i parametri principali; nel seguito ne descriviamo la struttura e le scelte progettuali.

La **rete neurale Base** è un percettrone multistrato volutamente semplice. Riceve in ingresso le dieci feature standardizzate e le elabora attraverso due strati nascosti completamente connessi, il primo da 32 neuroni e il secondo da 16, entrambi con attivazione ReLU; un neurone finale con attivazione sigmoide produce la probabilità che l'evento sia un gamma. L'addestramento usa l'ottimizzatore Adam con tasso di apprendimento 10^{-3} e la funzione di perdita binary cross-entropy, per 75 epoche con batch da 32 campioni. È importante sottolineare che questa rete non ha alcuna forma di regolarizzazione: la consideriamo il nostro modello “ingenuo”, quello che ci aspettiamo più esposto al rumore proprio perché nulla gli impedisce di adattarsi anche ai dati corrotti.

La **rete neurale Pro** parte dalla stessa architettura a due strati nascosti (32 e 16 neuroni) ma vi aggiunge quattro difese contro l'overfitting, ciascuna con una motivazione precisa. La regolarizzazione $L2$ (coefficiente 10^{-4}) sui pesi dei due strati nascosti penalizza i pesi grandi e quindi scoraggia soluzioni eccessivamente complesse. Il *dropout* (con probabilità 0,2), inserito dopo ciascuno strato nascosto, disattiva casualmente una frazione di neuroni a ogni passo di addestramento, costringendo la rete a non dipendere da singole unità e a costruire rappresentazioni più ridondanti. Il *class weight* bilanciato compensa lo sbilanciamento tra le classi assegnando un peso maggiore agli adroni, la classe minoritaria, così che gli errori su di essi non vengano trascurati a favore della classe abbondante. Infine l'*early stopping* monitora la perdita sul validation set e interrompe l'addestramento (fino a un massimo di 150 epoche) quando essa smette di migliorare per 15 epoche consecutive, ripristinando i pesi del momento migliore: è il meccanismo che ci aspettiamo decisivo contro il rumore sulle etichette, perché impedisce alla rete di continuare ad apprendere ben oltre il punto di generalizzazione ottimale.

Il **Random Forest** è un insieme di 200 alberi decisionali, ciascuno addestrato su un sottoinsieme casuale di dati e di feature. La sua robustezza nasce dalla media tra molti alberi: gli errori idiosincratici di un singolo albero tendono a cancellarsi nell'aggregazione per voto. A differenza delle reti, non procede per epoche, quindi per esso non esistono curve di addestramento e ci limiteremo a mostrarne la matrice di confusione e il report di classificazione.

Tabella 2: Parametri principali dei tre modelli confrontati.

Modello	Architettura / struttura	Regolarizzazione / iperparametri
NN Base	Dense(32)→Dense(16)→sigmoide	nessuna; Adam 10^{-3} , 75 epoche, batch 32
NN Pro	Dense(32)→Dense(16)→sigmoide (stessa architettura)	$L2\ 10^{-4}$, dropout 0,2, class weight bilanciato, early stopping (patience 15), max 150 epoche
Random Forest	200 alberi decisionali	aggregazione per voto, <code>random_state=42</code>

Per valutare i modelli abbiamo usato quattro metriche complementari, tutte calcolate sul test set pulito applicando alla probabilità prevista la soglia di 0,5. L'**accuratezza** è immediata ma, come abbiamo visto, poco affidabile con classi sbilanciate; l'**F1 macro** e la **recall macro** mediano le prestazioni sulle due classi dando lo stesso peso ad adroni e gamma, e quindi penalizzano un modello che trascuri la classe minoritaria; l'**AUC**

misura la capacità di ordinare correttamente i campioni indipendentemente dalla soglia scelta. Nelle tabelle dei risultati riportiamo accuratezza, F1 macro e AUC, che riteniamo la combinazione più informativa per questo problema.

7 Baseline su Dati Puliti

Prima di introdurre qualunque rumore abbiamo addestrato i tre modelli sui dati puliti, ottenendo il riferimento (livello 0%) rispetto al quale misureremo ogni successivo degrado. La Tabella 3 riassume le metriche di test.

Tabella 3: Metriche di test dei tre modelli sui dati puliti (riferimento a 0% di rumore).

Modello	Accuratezza	F1 macro	AUC
NN Base	0,8678	0,8517	0,9214
NN Pro	0,8601	0,8468	0,9238
Random Forest	0,8785	0,8629	0,9300

Osservazione

Sui dati puliti i tre modelli sono quasi equivalenti (accuratezza 86–88%, AUC 0,92–0,93), tutti ben al di sopra del 65% del classificatore banale stimato nella Sezione 4. La rete Pro sacrifica un soffio di accuratezza in cambio di un AUC già superiore: un investimento in stabilità che ci attendiamo di veder ripagato sotto rumore.

I tre modelli partono dunque da prestazioni molto vicine. Il Random Forest è leggermente il migliore su tutte le metriche, mentre la rete Pro mostra un’accuratezza appena inferiore alla Base: questo è atteso e non preoccupante, perché la regolarizzazione sacrifica un po’ di adattamento ai dati puliti in cambio di maggiore stabilità. È significativo, però, che l’AUC della rete Pro sia già superiore a quella della Base, segno che l’ordinamento delle probabilità è più solido nonostante l’accuratezza leggermente più bassa.

Le matrici di confusione dei tre modelli (Figura 5) mostrano un comportamento tipico con classi sbilanciate: tutti riconoscono molto bene i gamma, la classe abbondante, e commettono proporzionalmente più errori sugli adroni. Le curve di addestramento delle due reti (Figura 6) confermano invece una convergenza regolare e un divario contenuto tra train e validation: sui dati puliti nessuna delle due reti soffre di overfitting apprezzabile. È un punto di riferimento importante, perché vedremo che è proprio sotto rumore sulle etichette che questo divario potrà esplodere.

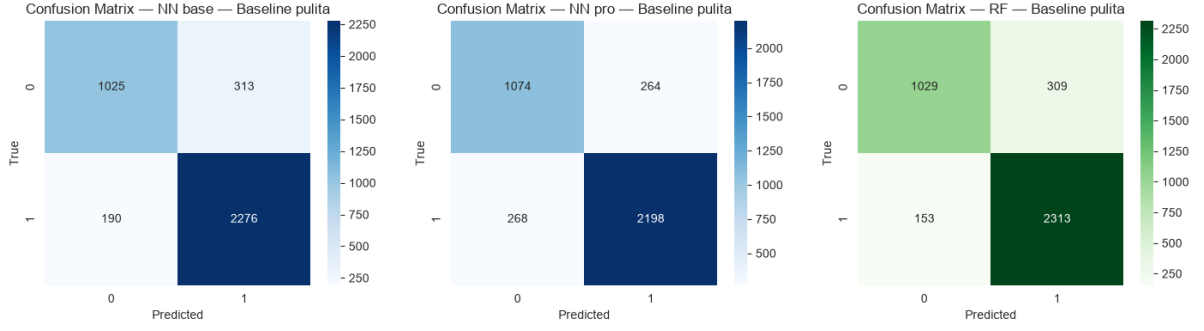


Figura 5: Matrici di confusione sui dati puliti: rete Base, rete Pro e Random Forest (da sinistra a destra). Tutti riconoscono bene i gamma; gli errori si concentrano sugli adroni, la classe minoritaria.

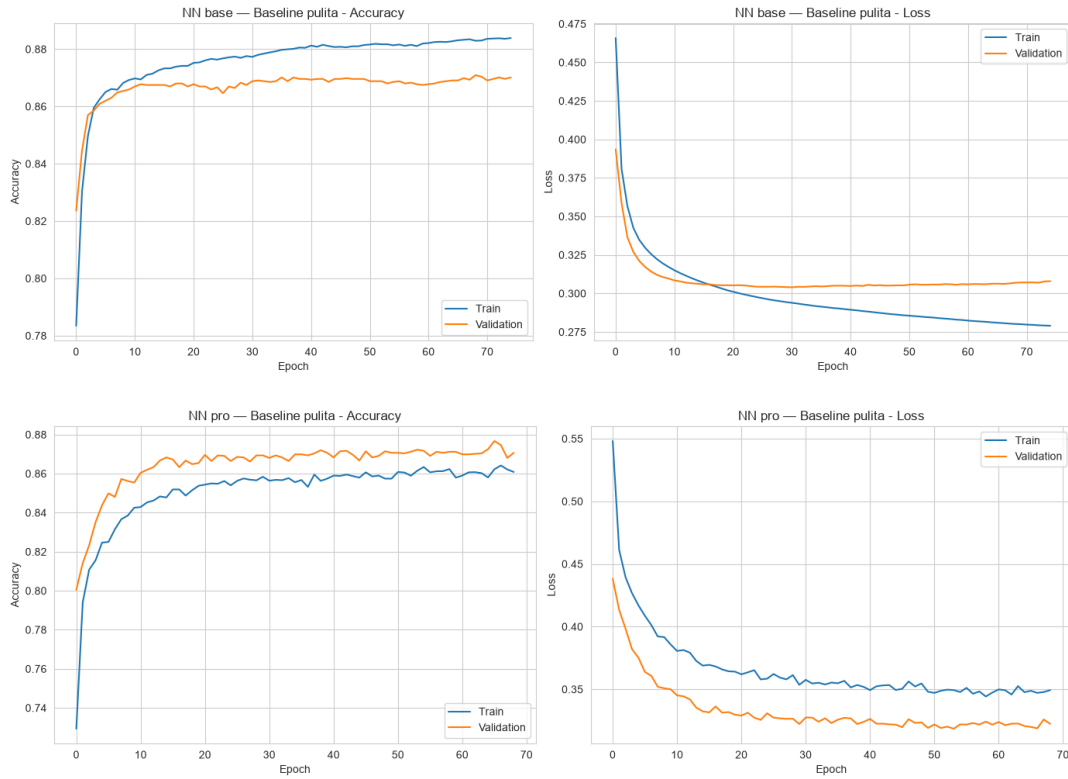


Figura 6: Curve di accuratezza e loss (train vs validation) sui dati puliti: rete Base (in alto) e rete Pro (in basso). In entrambe la convergenza è regolare e il divario train/validation è contenuto.

Sui dati puliti abbiamo infine calcolato l'importanza delle feature secondo il Random Forest (Figura 7). La gerarchia conferma in modo quantitativo quanto avevamo osservato nei boxplot (Figura 3): **fAlpha** è di gran lunga la feature più importante, con un valore vicino a 0,24, più del doppio della seconda; seguono **fLength**, **fWidth** e **fSize**, mentre i momenti trasversali contribuiscono pochissimo. Questa concentrazione dell'informazione in poche feature è un dato che si rivelerà centrale: definirà l'ordine con cui, nello scenario di feature mancanti, simuleremo il guasto dei sensori, partendo da quelli più preziosi.

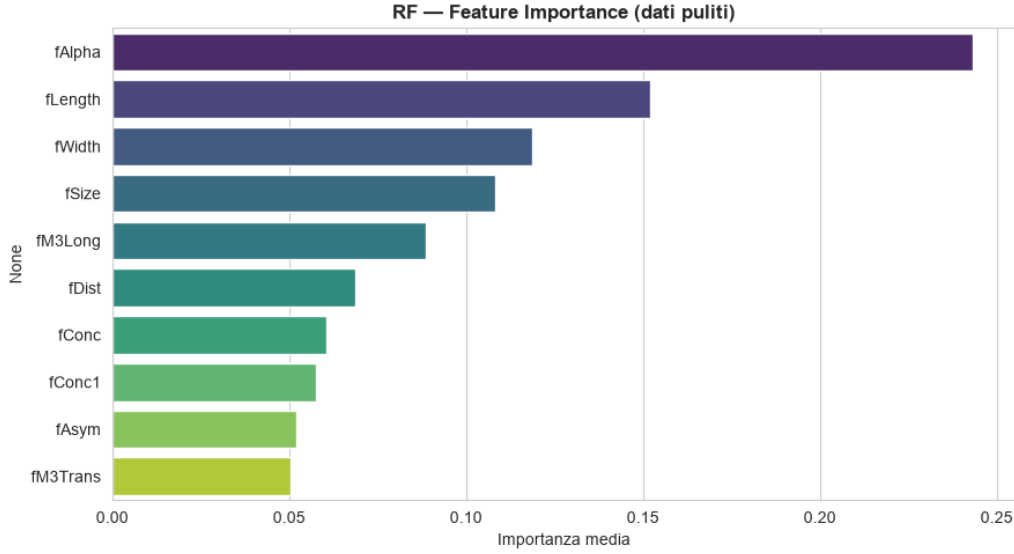


Figura 7: Importanza delle feature secondo il Random Forest sui dati puliti. **fAlpha** domina nettamente, coerentemente con la separabilità osservata nei boxplot (Figura 3).

8 Metodologia di Iniezione del Rumore

Avendo stabilito il riferimento sui dati puliti e la gerarchia di importanza delle feature, possiamo ora descrivere con precisione come “sporchiamo” i dati. Presentiamo qui la metodologia perché alcuni scenari, in particolare quello delle feature mancanti, si appoggiano sull’importanza delle feature appena calcolata nella baseline (Sezione 7). Abbiamo definito sei tipi di rumore, ciascuno pensato per riprodurre un guasto realistico di una pipeline dati; la Tabella 4 li riassume.

Tabella 4: I sei tipi di rumore studiati e il guasto reale che simulano.

Tipo di rumore	Cosa simula
Feature noise	misurazioni imprecise (rumore gaussiano sulle feature)
Label noise random	errori di etichettatura casuali e uniformi
Label noise borderline	errori di etichettatura sui campioni ambigui
Missing values	celle mancanti (NaN) da imputare
Outlier	valori estremi anomali da trattare
Missing feature	uno o più sensori guasti (feature non disponibili)

8.1 Protocollo comune e livelli di intensità

Un principio condiviso da tutti gli esperimenti è che il rumore viene applicato al *solo* training set, mentre validation e test restano puliti; fa eccezione il solo scenario dei valori mancanti, in cui il disturbo viene applicato anche a validation e test per realismo e poi rimosso tramite imputazione. Questa scelta è deliberata: vogliamo misurare quanto un modello addestrato su dati sporchi sappia comunque generalizzare a dati corretti, che è la situazione tipica in cui ci si affida a un modello sperando che “filtri” il rumore visto in addestramento. Per ciascun tipo di rumore l’esperimento riaddestra *tutti e tre* i modelli

sul riferimento pulito e poi sui tre livelli di intensità, calcolando ogni volta le metriche sullo stesso test set pulito.

Ogni rumore è stato applicato a tre livelli — 10%, 25% e 40% — oltre al riferimento a 0%. Il significato del “livello” dipende dalla natura del disturbo, come spieghiamo qui di seguito descrivendo i sei meccanismi uno per uno.

8.2 Descrizione dei sei meccanismi

Feature noise (rumore gaussiano). Aggiungiamo a ciascuna feature del training un rumore gaussiano a media nulla e deviazione standard pari al livello scelto. Poiché i dati sono standardizzati (Sezione 5), un livello di 0,40 corrisponde a un rumore con ampiezza pari al 40% della deviazione standard della feature, una quantità interpretabile e identica per tutte. Simula sensori imprecisi che restituiscono misure leggermente sbagliate.

Label noise random. Invertiamo l’etichetta (da 0 a 1 e viceversa) di una frazione di campioni del training pari al livello, scelti in modo del tutto casuale e uniforme. Simula errori di annotazione distribuiti senza alcuno schema particolare.

Label noise borderline. Anche qui invertiamo una frazione di etichette pari al livello, ma la selezione non è casuale: calcoliamo i centroidi delle due classi nello spazio standardizzato e, per ogni campione, la distanza dal centroide della classe *opposta*; invertiamo poi le etichette dei campioni con distanza minore, cioè quelli geometricamente più vicini all’altra classe e dunque più ambigui. È un disturbo deliberatamente più “insidioso” del rumore casuale, perché corrompe proprio gli esempi vicini al confine decisionale, là dove la separazione tra le classi è più delicata; ci aspettiamo quindi che, a parità di percentuale, danneggi più del rumore casuale.

Missing values. Sostituiamo con un valore mancante (NaN) ogni cella del dataset con probabilità pari al livello, in modo indipendente. Poiché né le reti né il Random Forest accettano valori mancanti, è necessaria un’*imputazione*; abbiamo confrontato due strategie, sostituire i NaN con la media oppure con la mediana della rispettiva feature. In coerenza con il principio di non far trapelare informazione, lo strumento di imputazione è stato calcolato solo sul training e applicato con gli stessi valori a validation e test. Come variante canonica per i confronti aggregati abbiamo scelto la mediana, motivati dall’asimmetria delle distribuzioni osservata nell’analisi esplorativa (Figura 2), che la rende un riassunto più stabile della media.

Outlier. Selezioniamo una frazione di campioni del training pari al livello e ne sostituiamo tutti i valori con quantità estreme, pari a ± 5 deviazioni standard con segno casuale per ciascuna cella. Anche qui abbiamo confrontato tre modi di reagire: *lasciarli* inalterati, *winsorizzarli* (troncare il 5% delle code di ogni feature, attenuando gli estremi senza eliminarli) oppure *imputarli* (marcare come mancanti i valori con punteggio z in modulo superiore a 4 e sostituirli con la mediana). Come variante canonica abbiamo adottato la winsorizzazione, che i dati stessi giustificheranno come buon compromesso tra semplicità ed efficacia.

Missing feature (sensore guasto). Azzeriamo interamente una o più feature, simulando il guasto dei sensori che le producono. Questo scenario non ha un livello percentuale naturale, perciò lo abbiamo mappato sulla rimozione progressiva delle feature più importanti secondo il Random Forest (Figura 7): il livello 10% azzerla la sola feature più importante (**fAlpha**), il 25% le prime tre e il 40% le prime cinque. La rimozione è applicata in modo coerente a training, validation e test. La scelta di partire dalle feature più informative rende lo scenario volutamente pessimistico: stiamo simulando il guasto dei sensori *più* preziosi, non di quelli marginali.

9 Risultati per Tipo di Rumore

In questa sezione analizziamo, un disturbo alla volta, come le prestazioni si degradano al crescere del rumore. Per ogni rumore riportiamo la tabella completa delle metriche ai quattro livelli e i grafici di confronto tra i tre modelli su accuratezza e F1 macro, accompagnati, dove utile, dalle curve di addestramento e dalle matrici di confusione nel caso più estremo. Il meccanismo di ciascun disturbo è quello descritto nella Sezione 8.

9.1 Feature Noise

Il rumore gaussiano sulle feature produce un degrado graduale e contenuto. Come mostrato nella Tabella 5, anche al 40% tutti i modelli restano sopra l'83% di accuratezza, con un calo di appena tre-quattro punti rispetto al riferimento, e le curve di accuratezza dei tre modelli (Figura 8, a sinistra) restano vicine e quasi piatte. L'andamento dell'F1 macro (Figura 8, a destra) conferma lo stesso quadro.

Tabella 5: Feature noise: metriche al variare del livello di rumore (test set).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8657	0.85	0.9187	0.8636	0.85	0.9204	0.8646	0.84	0.9222
25%	0.8520	0.83	0.9074	0.8465	0.83	0.9071	0.8494	0.82	0.9041
40%	0.8328	0.80	0.8941	0.8417	0.82	0.8940	0.8318	0.80	0.8913

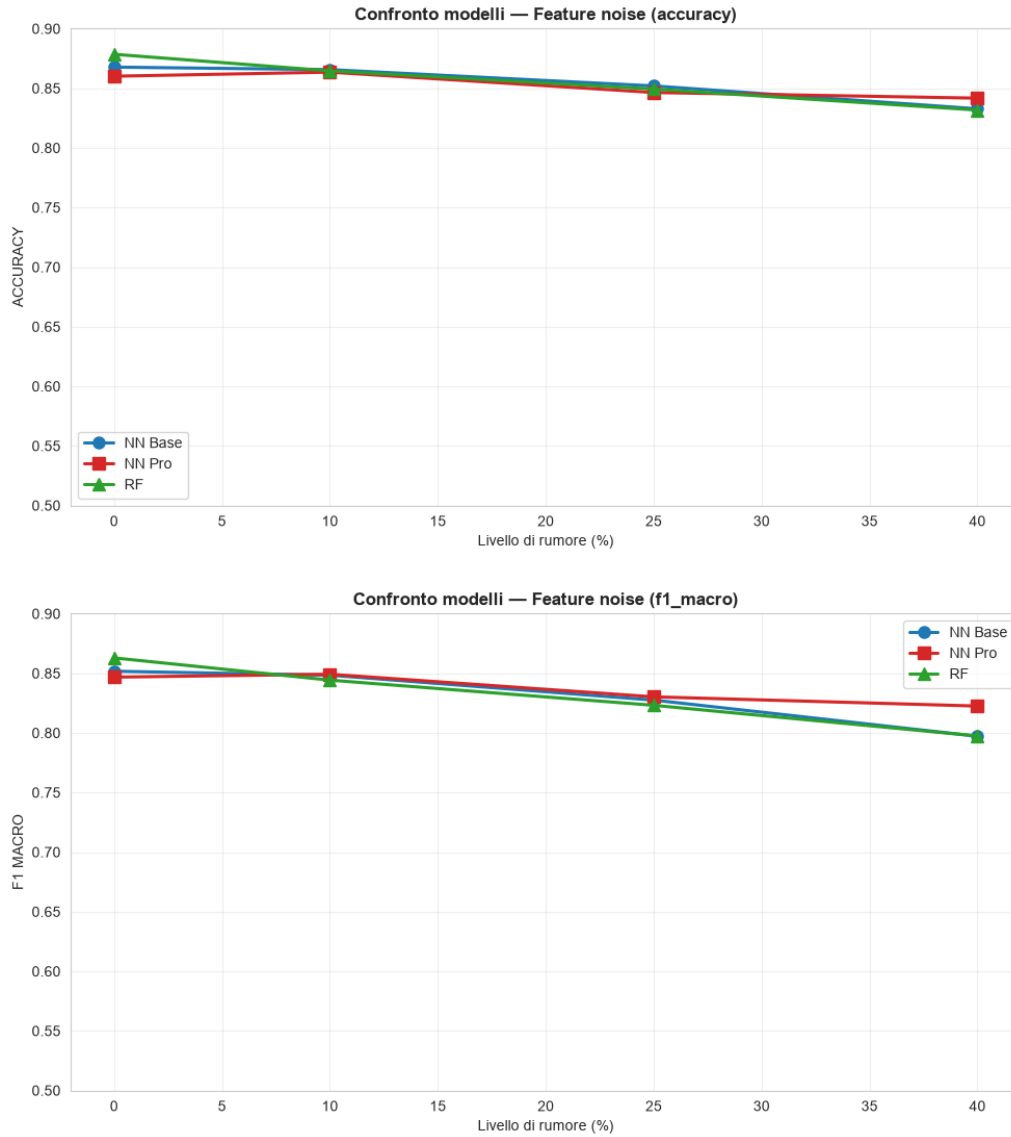


Figura 8: Feature noise: accuratezza (in alto) e F1 macro (in basso) dei tre modelli al crescere del rumore. Il degrado è lieve e le tre curve restano vicine.

L'aspetto più interessante emerge al livello più alto: a 40% la rete Pro (0,8417) supera sia la Base (0,8328) sia il Random Forest (0,8318). È la prima conferma concreta della nostra ipotesi sulla regolarizzazione: di fronte a feature rumorose, L2 e dropout impediscono alla rete di inseguire le fluttuazioni casuali del segnale e la mantengono più stabile, mentre la rete Base, priva di freni, comincia ad adattarsi al rumore e perde leggermente terreno. Il confronto tra le curve di addestramento al 40% (Figura 9) rende visibile questa differenza: la rete Base mantiene un piccolo divario tra train e validation, mentre la rete Pro procede con le due curve praticamente sovrapposte, segno che non sta adattandosi al rumore. Il fatto che il calo complessivo resti comunque modesto si spiega con la ridondanza tra le feature osservata nell'analisi esplorativa (Figura 4): il rumore aggiunto a una feature è in parte compensato dall'informazione correlata presente nelle altre.

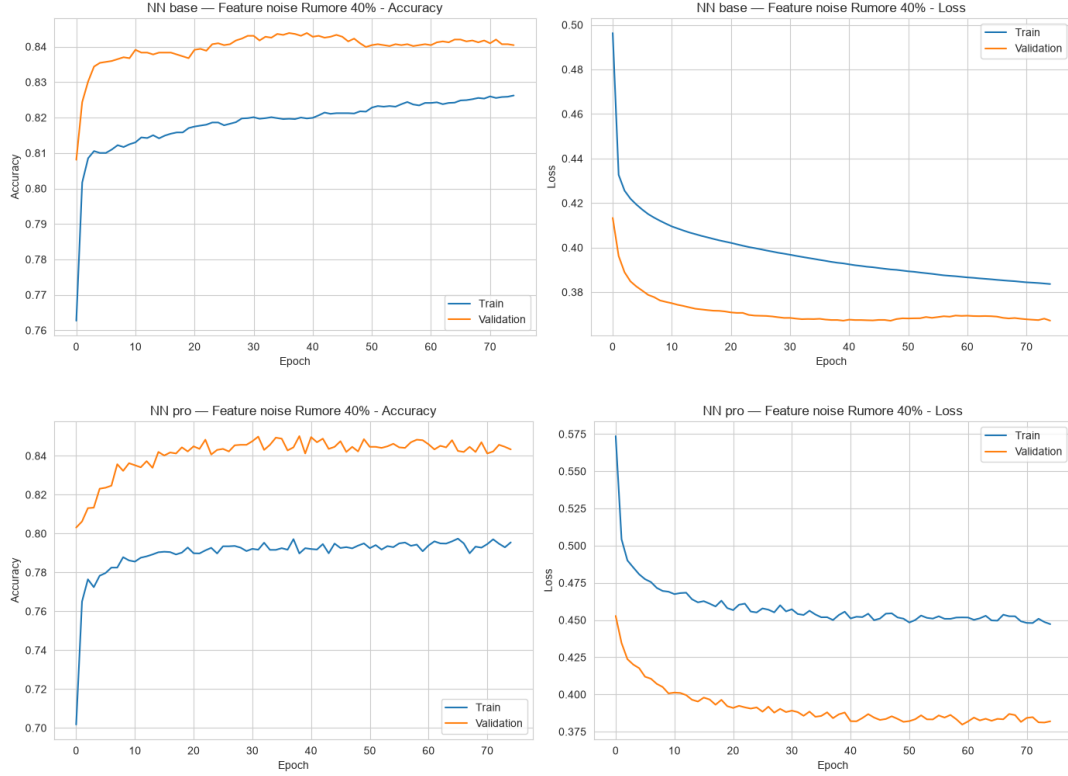


Figura 9: Feature noise al 40%: curve di addestramento della rete Base (in alto) e della rete Pro (in basso). Nella rete Pro train e validation restano sovrapposte: la regolarizzazione evita l’adattamento al rumore.

Questo degrado contenuto, però, è in realtà *asimmetrico* tra le due classi. Poiché il dataset è sbilanciato (Sezione 4), l’accuratezza complessiva resta alta anche quando il modello comincia a sacrificare la classe minoritaria: sotto feature noise al 40% la recall degli adroni della rete Base scende da 0,77 a 0,59, mentre quella dei gamma addirittura sale da 0,92 a 0,97. Il modello, in difficoltà, “si rifugia” nella classe maggioritaria, classificando un numero crescente di adroni come gamma. È qui che si apprezza il *class weight* bilanciato della rete Pro (Sezione 6): mantiene la recall degli adroni a 0,73, contro lo 0,59 della Base e lo 0,60 del Random Forest, ed è proprio questa protezione della classe rara a spiegare il suo F1 macro più alto a parità di accuratezza. La progressione delle matrici di confusione della rete Base (Figura 10) rende visibile il fenomeno: livello dopo livello la colonna delle predizioni “gamma” si ingrossa, perché sempre più adroni (riga 0) vengono erroneamente assegnati al segnale, mentre i gamma (riga 1) restano riconosciuti.

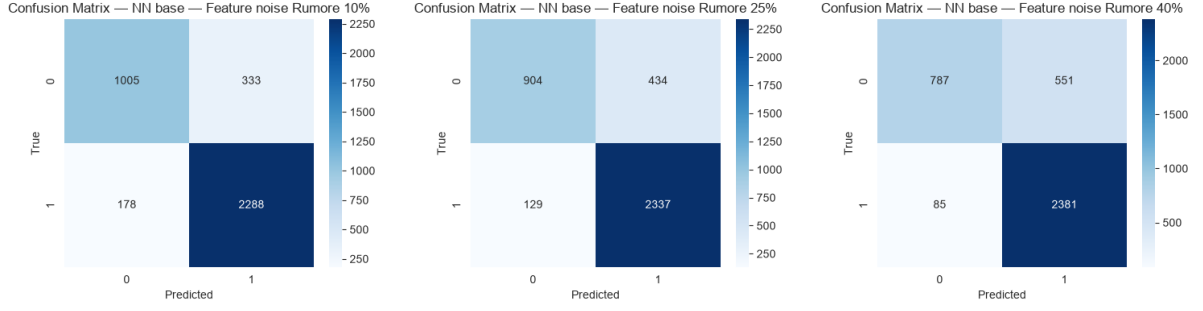


Figura 10: Feature noise, rete Base: matrici di confusione al 10%, 25% e 40% (da sinistra a destra). Gli adroni (riga 0) vengono classificati come gamma con frequenza crescente, mentre i gamma (riga 1) restano riconosciuti.

9.2 Label Noise Random

Con l'inversione casuale delle etichette il quadro cambia radicalmente rispetto al feature noise. La Tabella 6 mostra che fino al 25% il degrado resta gestibile, ma al 40% le prestazioni crollano per due dei tre modelli: la rete Base scende a 0,7419 di accuratezza e a 0,69 di F1 macro, e il Random Forest addirittura a 0,7179. La rete Pro, invece, regge molto meglio, restando a 0,8044 di accuratezza e 0,79 di F1 macro. La Figura 11 visualizza con chiarezza questo divario: al 40% la curva della rete Pro si stacca verso l'alto mentre le altre due precipitano, e lo stesso accade per l'F1 macro.

Tabella 6: Label noise random: metriche al variare del livello di rumore (test set).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8623	0.84	0.9128	0.8570	0.84	0.9200	0.8767	0.86	0.9214
25%	0.8410	0.82	0.8882	0.8378	0.83	0.9081	0.8399	0.82	0.8884
40%	0.7419	0.69	0.7513	0.8044	0.79	0.8677	0.7179	0.70	0.7673

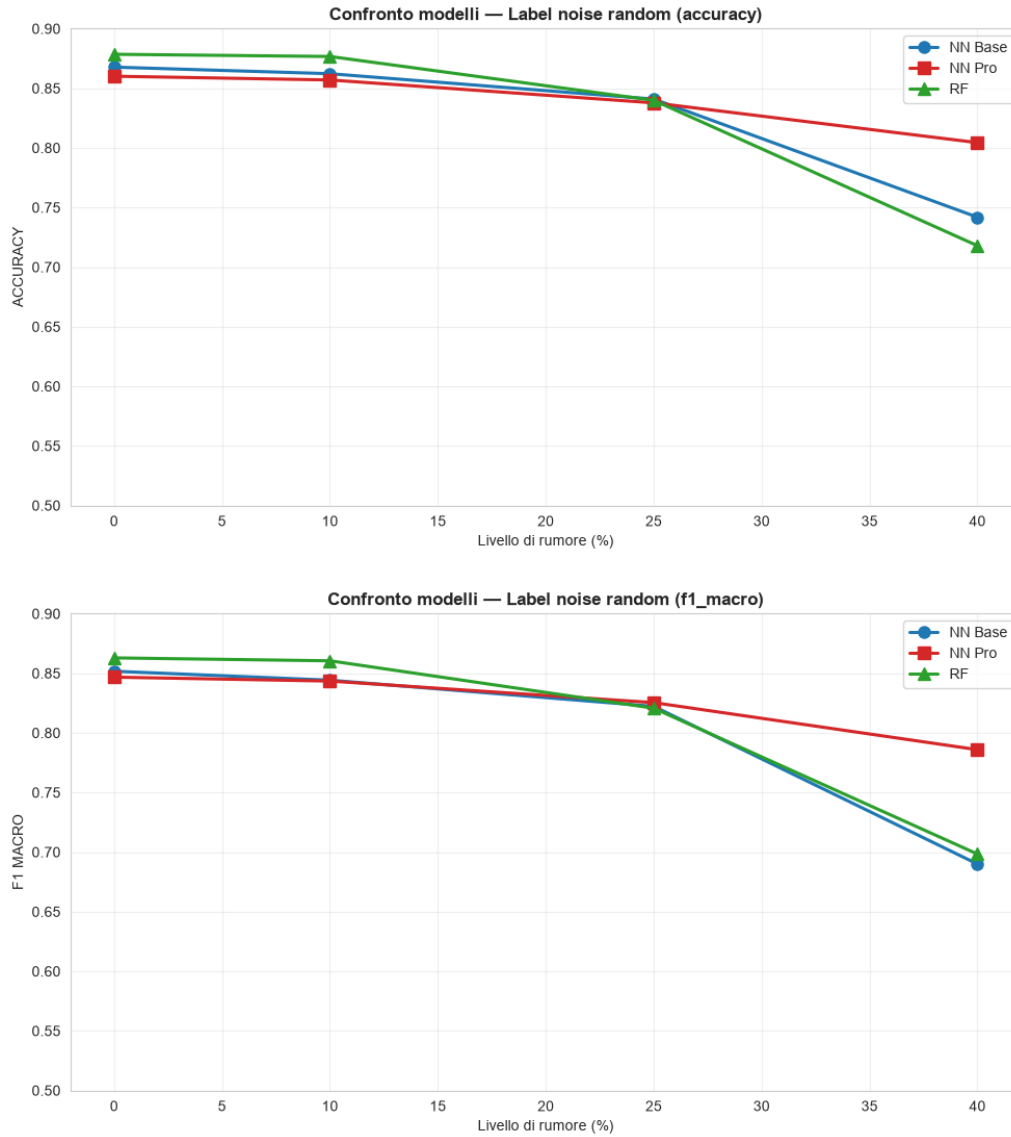


Figura 11: Label noise random: accuratezza (in alto) e F1 macro (in basso) dei tre modelli. Al 40% la rete Pro si stacca nettamente verso l'alto, mentre Base e Random Forest crollano.

La spiegazione di questo divario sta nelle curve di addestramento al 40%, che riteniamo la coppia di figure più istruttiva dell'intero studio. Nella rete Base (Figura 12, a sinistra) l'accuratezza sul training continua a salire fino a superare l'88%, mentre l'accuratezza di validation scende e la perdita di validation cresce in modo monotono: la rete sta letteralmente memorizzando le etichette sbagliate. Nella rete Pro (Figura 12, a destra) accade l'opposto: l'accuratezza di training resta bassa, attorno al 57%, perché viene misurata sulle etichette corrotte e la rete, grazie all'early stopping, si ferma prima di poterle memorizzare; di conseguenza l'accuratezza di validation, calcolata su etichette pulite, resta alta attorno all'82% e la perdita di validation *diminuisce*. È esattamente la situazione in cui l'arresto anticipato fa la differenza, impedendo alla rete di apprendere il rumore oltre il punto di generalizzazione ottimale.

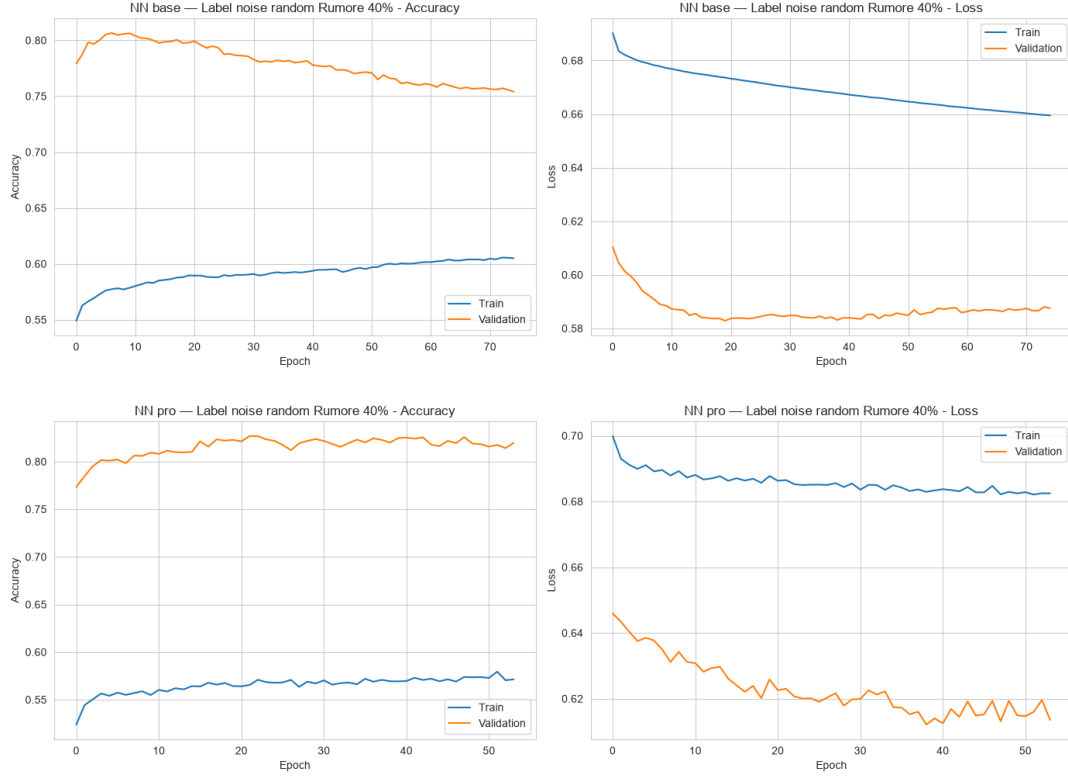


Figura 12: Label noise random al 40%: curve della rete Base (in alto) e della rete Pro (in basso). La Base memorizza le etichette corrotte (train sale, validation scende); la Pro, fermata dall'early stopping, mantiene alta la validation.

Il Random Forest, pur essendo un insieme di alberi, non dispone di un meccanismo analogo di arresto basato su un insieme pulito e finisce per incorporare gran parte degli errori, risultando qui il modello meno robusto. La sua matrice di confusione al 40% (Figura 13) mostra in modo netto la conseguenza: la capacità di riconoscere gli adroni si riduce sensibilmente e gli errori si distribuiscono su entrambe le classi.

Anche qui l'impatto sulle classi è diseguale, ma con una sfumatura propria del rumore sulle etichette. Nella rete Base la recall degli adroni crolla da 0,77 a 0,47 mentre i gamma restano a 0,89: come per il feature noise, è la classe minoritaria a pagare il prezzo più alto, e ancora una volta il *class weight* della rete Pro lo contiene (0,73 sugli adroni). Il Random Forest fa eccezione, perché vede ridursi la recall di *entrambe* le classi (adroni 0,66, gamma 0,75): avendo assorbito le etichette corrotte in modo più uniforme, il suo errore non si concentra su una sola classe ma le coinvolge entrambe.

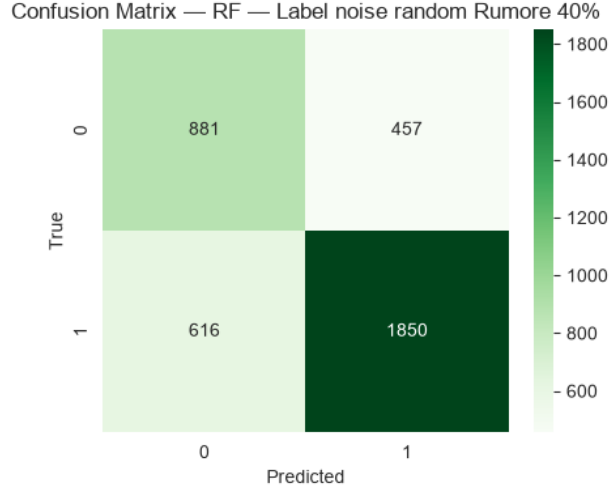


Figura 13: Label noise random al 40%: matrice di confusione del Random Forest. Gli errori aumentano su entrambe le classi: il modello ha assorbito gran parte delle etichette corrotte.

9.3 Label Noise Borderline

Invertire le etichette dei campioni più ambigui si conferma il disturbo più dannoso in assoluto. La Tabella 7 mostra cali già severi al 25% e drammatici al 40%, dove tutti i modelli scendono attorno al 60% di accuratezza — un valore prossimo a quello del classificatore banale stimato nell’analisi esplorativa (Sezione 4), segno che gran parte della capacità discriminante è andata perduta. La Figura 14 mostra curve molto più ripide di quelle del rumore casuale.

Tabella 7: Label noise borderline: metriche al variare del livello di rumore (test set).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8252	0.80	0.8680	0.8352	0.82	0.8915	0.8383	0.81	0.8886
25%	0.7326	0.71	0.7879	0.7403	0.72	0.8265	0.7387	0.72	0.8043
40%	0.6002	0.59	0.6656	0.6354	0.62	0.7084	0.6094	0.60	0.6774

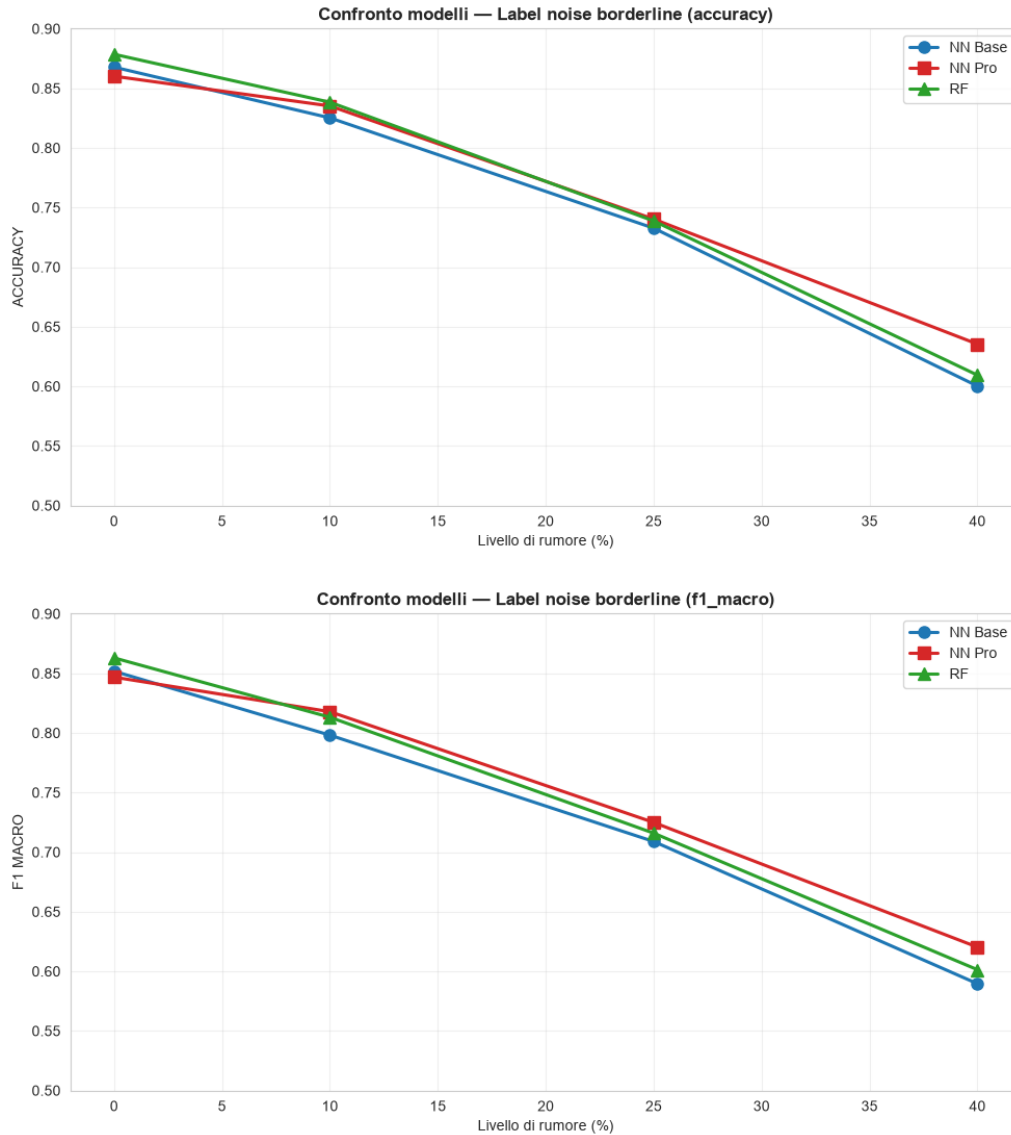


Figura 14: Label noise borderline: accuratezza (in alto) e F1 macro (in basso). Il crollo è molto più ripido del rumore casuale; al 40% tutti i modelli convergono attorno al 60%.

Il confronto diretto con il rumore casuale è illuminante: a parità di percentuale, il borderline degrada molto di più. Al 40%, ad esempio, l'accuratezza della rete Base passa da 0,7419 con rumore casuale a 0,6002 con rumore borderline. La ragione è che invertire le etichette degli esempi ambigui non si limita ad aggiungere “confusione” uniforme: sposta attivamente il confine decisionale, perché proprio quei campioni, vicini alla separazione tra le classi, sono i più influenti nel determinarne la posizione. Una conseguenza importante è che qui il vantaggio della regolarizzazione si assottiglia. Le curve di addestramento al 40% (Figura 15) lo mostrano bene: anche la rete Pro, che sul rumore casuale manteneva alta la validation, vede ora l'accuratezza di validation scendere ben sotto il 65%. Quando il rumore falsifica sistematicamente il confine, nemmeno l'arresto anticipato può recuperare un'informazione che è stata corrotta nel punto più sensibile; la rete Pro conserva solo un piccolo margine (0,6354 contro 0,6002 della Base).

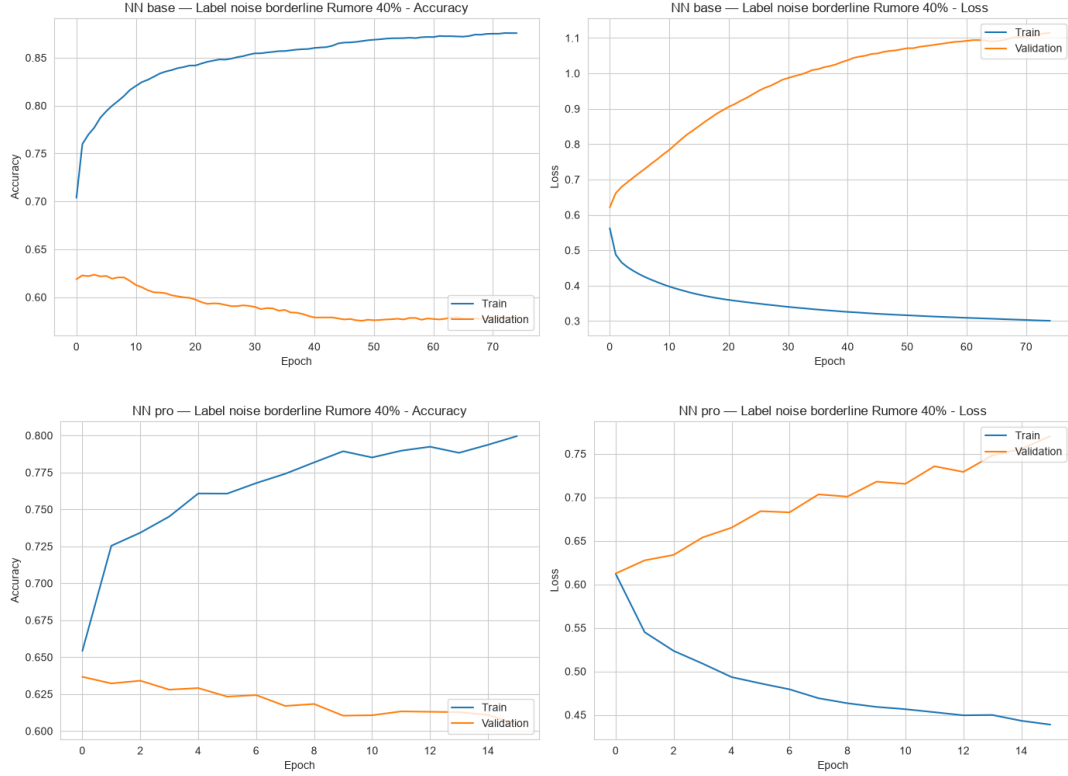


Figura 15: Label noise borderline al 40%: curve della rete Base (in alto) e della rete Pro (in basso). A differenza del rumore casuale, anche la validation della rete Pro crolla: il confine decisionale è stato spostato.

Le matrici di confusione al 40% delle reti Pro e del Random Forest (Figura 16) confermano l'entità del danno: gli errori sono ormai diffusi su entrambe le classi e il modello fatica a separare gamma e adroni, esattamente come ci si attende quando il segnale di addestramento è stato corrotto proprio nella regione di confine. È significativo che, a differenza di tutti gli altri rumori, qui il degrado sia *simmetrico* tra le classi: al 40% la recall di adroni e gamma scende per ogni modello attorno a 0,60 (per la rete Base 0,62 e 0,59 rispettivamente). Il modello non scivola verso la classe maggioritaria come accadeva col feature noise, ma perde potere discriminante su entrambe; ed è proprio per questo che il riequilibrio delle classi della rete Pro non offre quasi alcun vantaggio, dato che il problema non è più la proporzione tra le classi ma la posizione stessa del confine, ormai falsificata.

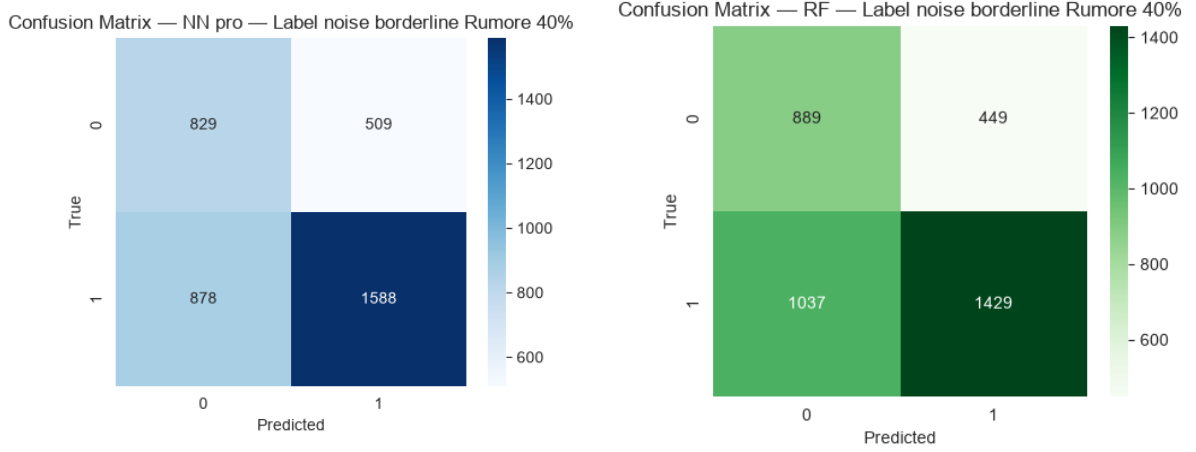


Figura 16: Label noise borderline al 40%: matrici di confusione della rete Pro (sinistra) e del Random Forest (destra). Gli errori sono diffusi su entrambe le classi.

9.4 Missing Values

Con i valori mancanti il degrado è moderato e regolare, a patto di imputare. La Tabella 8 riporta i risultati con la mediana, la variante canonica: al 40% i modelli restano tra il 76% e l'81% di accuratezza, con il Random Forest (0,8068) nettamente il più resistente, come si vede dalla Figura 17.

Tabella 8: Missing values (imputazione con mediana): metriche al variare del tasso di valori mancanti.

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8423	0.82	0.8947	0.8375	0.82	0.9002	0.8559	0.84	0.9051
25%	0.8123	0.79	0.8645	0.7978	0.78	0.8649	0.8270	0.80	0.8750
40%	0.7792	0.74	0.8266	0.7650	0.75	0.8319	0.8068	0.78	0.8477

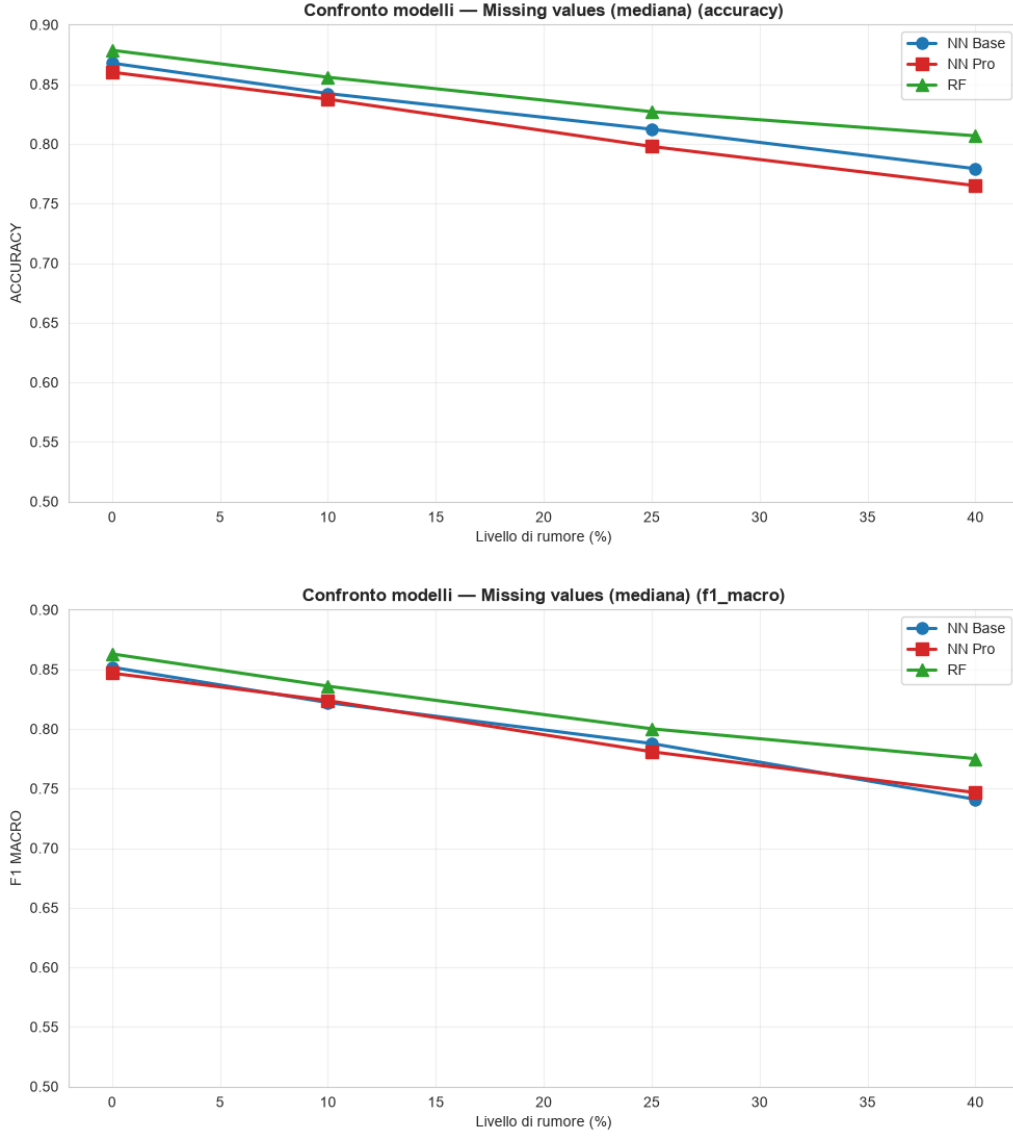


Figura 17: Missing values con imputazione mediana: accuratezza (in alto) e F1 macro (in basso). Il Random Forest mantiene un vantaggio costante a tutti i livelli.

Il confronto diretto tra media e mediana (Figura 18) conferma l'intuizione nata dall'osservazione delle distribuzioni asimmetriche nell'analisi esplorativa (Figura 2): la mediana è leggermente più robusta della media, perché non viene trascinata dalle code lunghe delle feature. Il vantaggio è modesto ma sistematico — ad esempio, per la rete Base al 40% l'accuratezza passa da 0,7713 con la media a 0,7792 con la mediana — e giustifica la nostra scelta della mediana come strategia di riferimento. La robustezza superiore del Random Forest si spiega con la sua natura ad alberi: le soglie di suddivisione sono meno sensibili a un singolo valore imputato di quanto non lo siano i pesi continui di una rete neurale.

Anche con i valori mancanti ritroviamo il degrado asimmetrico già visto sulle feature: al 40% la recall degli adroni della rete Base scende a 0,56 mentre quella dei gamma resta a 0,90, e ancora una volta il *class weight* della rete Pro contiene il calo sulla classe minoritaria (0,70 sugli adroni). Il Random Forest, oltre a essere il più accurato, è qui anche il più equilibrato tra le due classi, segno che l'imputazione con mediana ne preserva bene la capacità di riconoscere entrambi i tipi di evento.

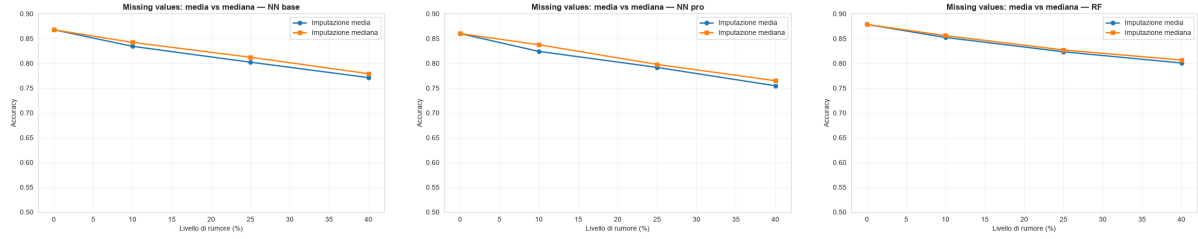


Figura 18: Missing values: confronto tra imputazione con media e con mediana per i tre modelli. La mediana mantiene un vantaggio piccolo ma costante, coerente con l'asimmetria delle feature.

Anche qui le curve di addestramento e le matrici di confusione al 40% (Figura 19) raccontano la stessa storia vista per gli altri disturbi sulle feature: l'addestramento resta regolare e privo di overfitting, ma gli errori si concentrano sugli adroni, che vengono in parte assorbiti dalla classe gamma. Il Random Forest, a destra, conferma di gestire meglio degli altri la classe minoritaria.

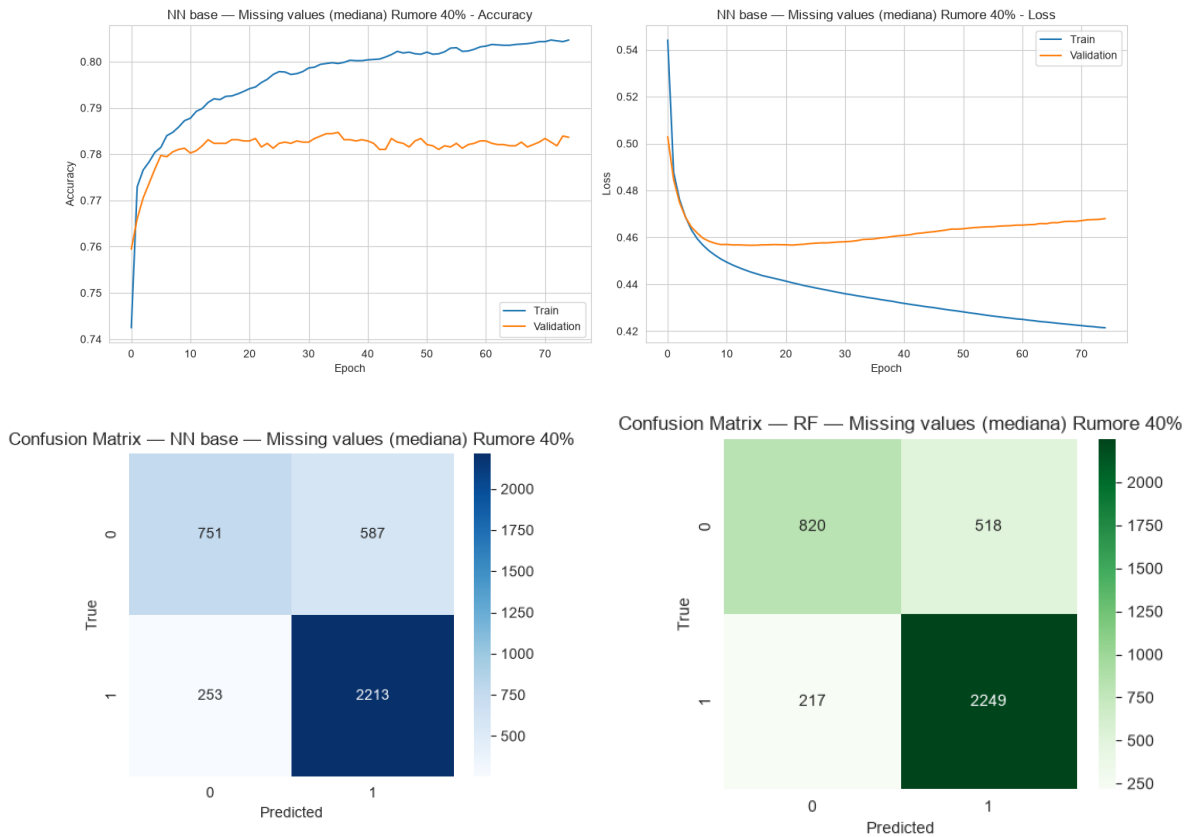


Figura 19: Missing values (mediana) al 40%: curve della rete Base (in alto), matrici di confusione della rete Base (in basso a sinistra) e del Random Forest (in basso a destra). L'addestramento è regolare; gli errori pesano soprattutto sugli adroni.

9.5 Outlier

Gli outlier artificiali sono, sorprendentemente, il disturbo meno dannoso di tutti. Con la winsorizzazione (Tabella 9) le prestazioni restano quasi invariate a ogni livello: al 40% tutti i modelli sono ancora attorno all'85–87% di accuratezza, con un calo trascurabile,

e le curve di confronto (Figura 20) sono praticamente piatte sia per l'accuratezza sia per l'F1 macro.

Tabella 9: Outlier (winsorizzazione): metriche al variare della frazione di campioni anomali.

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8657	0.85	0.9196	0.8583	0.84	0.9220	0.8788	0.86	0.9295
25%	0.8628	0.84	0.9160	0.8441	0.83	0.9173	0.8751	0.86	0.9267
40%	0.8615	0.84	0.9116	0.8541	0.84	0.9201	0.8736	0.86	0.9250

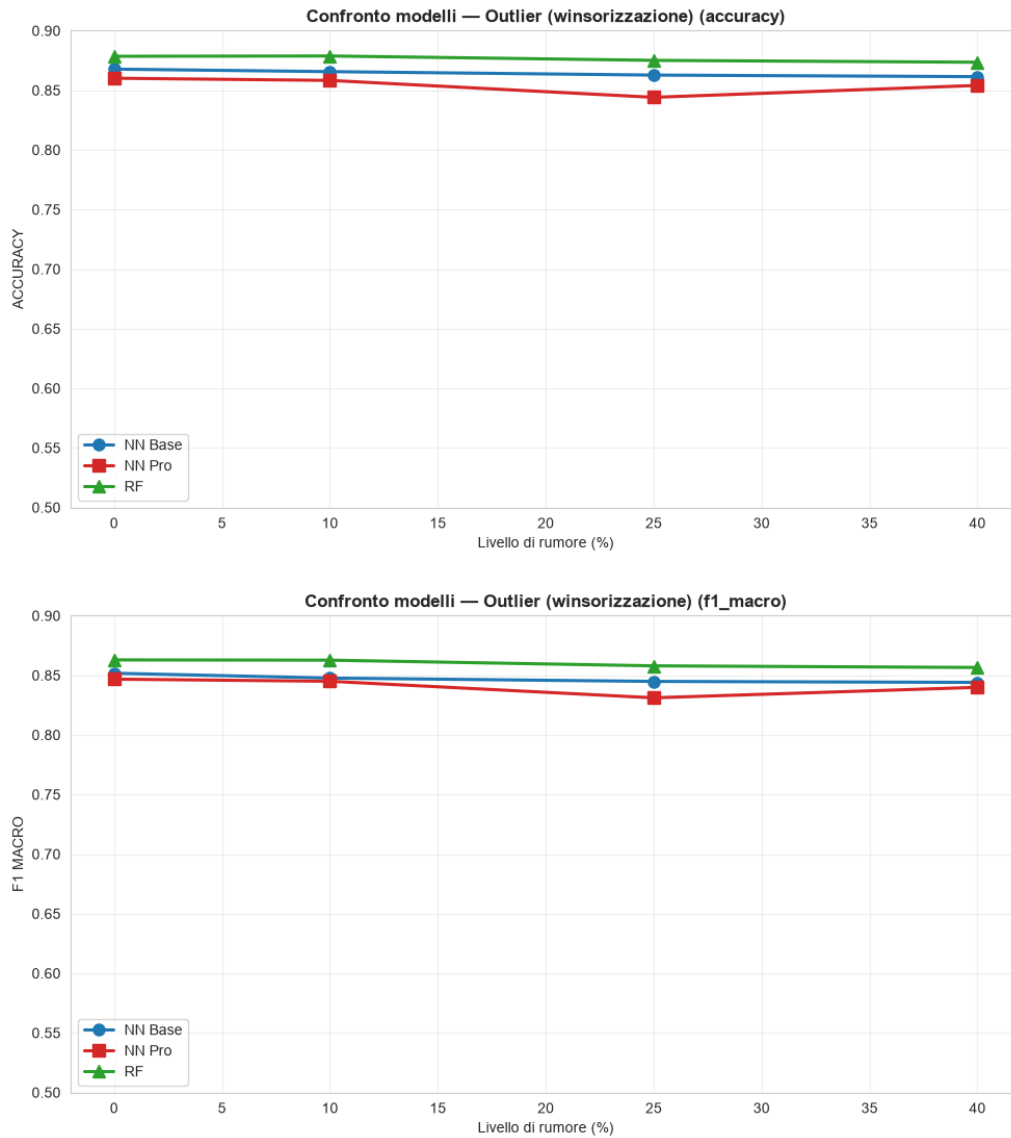


Figura 20: Outlier con winsorizzazione: accuratezza (in alto) e F1 macro (in basso). Le curve sono quasi piatte: il disturbo è ben assorbito dal trattamento.

Il confronto tra le tre strategie (Figura 21) mostra differenze minime ma coerenti: anche lasciare gli outlier inalterati danneggia poco i modelli, mentre la winsorizzazione

e l'imputazione recuperano quel poco margine perso. La spiegazione è che, lavorando su dati standardizzati (Sezione 5) e introducendo gli outlier solo nel training, il loro effetto sulle decisioni del modello è limitato; ciononostante, troncare le code rimane la scelta più prudente perché stabilizza l'addestramento senza eliminare campioni. Abbiamo quindi adottato la winsorizzazione come variante canonica, non perché sia indispensabile, ma perché offre il miglior compromesso tra semplicità e stabilità. Coerentemente con l'impatto trascurabile sull'accuratezza, anche le singole classi restano stabili: al 40% la recall degli adroni e dei gamma rimane sostanzialmente invariata rispetto ai dati puliti (per la rete Base 0,75 e 0,92). A differenza degli altri disturbi, qui non si osserva alcuno scivolamento verso la classe maggioritaria, a conferma che gli outlier, una volta trattati, non alterano l'equilibrio tra le classi.

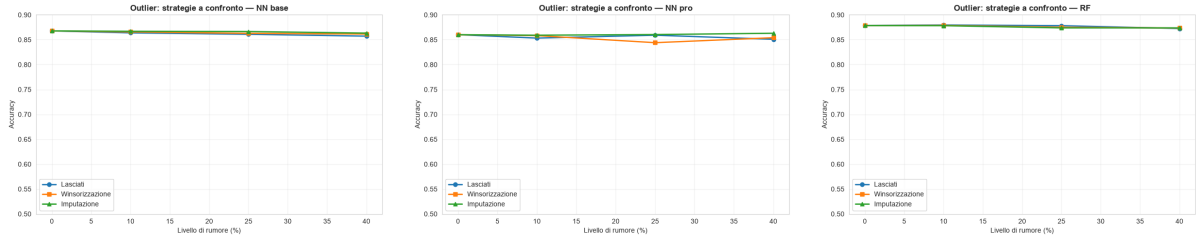


Figura 21: Outlier: confronto tra lasciarli inalterati, winsorizzazione e imputazione, per i tre modelli. Le differenze sono piccole; il trattamento recupera il modesto margine perso.

A conferma dell'impatto trascurabile, le matrici di confusione al 40% (Figura 22) sono praticamente indistinguibili da quelle sui dati puliti (Figura 5): il numero di adroni e gamma correttamente riconosciuti resta sostanzialmente lo stesso, segno che la winsorizzazione neutralizza quasi del tutto l'effetto dei valori estremi.

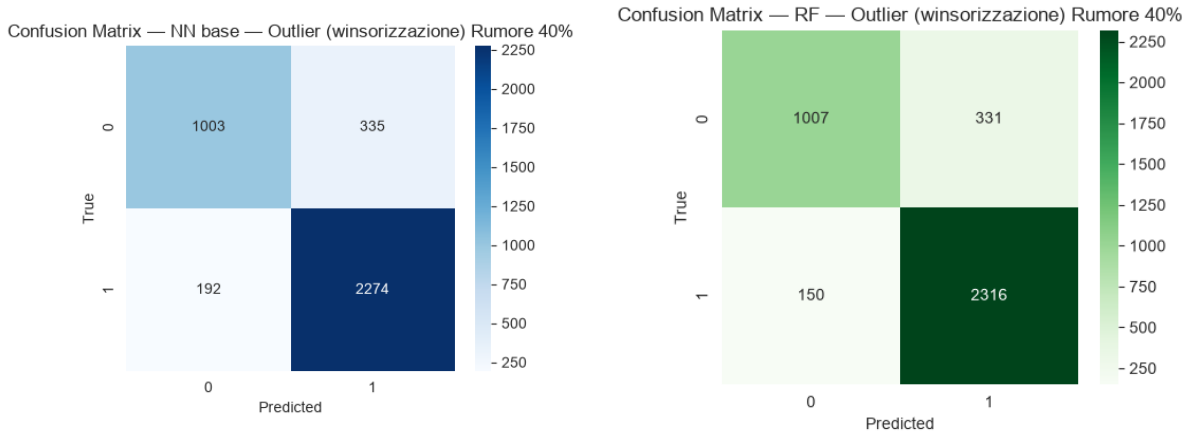


Figura 22: Outlier (winsorizzazione) al 40%: matrici di confusione della rete Base (sinistra) e del Random Forest (destra). Restano quasi identiche al caso pulito: il disturbo è ben assorbito dal trattamento.

9.6 Missing Feature (sensore guasto)

L'ultimo scenario azzerava intere feature secondo la gerarchia di importanza definita nella Sezione 8: $f\alpha$ al 10%, le prime tre al 25% e le prime cinque al 40%. La Tabella 10

mostra un degrado severo e quasi lineare (Figura 23): già al 10%, cioè rimuovendo la sola **fAlpha**, l'accuratezza scende di circa quattro-cinque punti, e al 40% tutti i modelli scendono attorno al 70–74%.

Tabella 10: Missing feature: metriche rimuovendo le feature più importanti (10%→top-1, 25%→top-3, 40%→top-5).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8215	0.79	0.8675	0.8052	0.79	0.8716	0.8294	0.80	0.8788
25%	0.7773	0.74	0.8007	0.7574	0.73	0.7991	0.7836	0.75	0.8195
40%	0.7303	0.67	0.7413	0.6995	0.67	0.7347	0.7363	0.68	0.7494

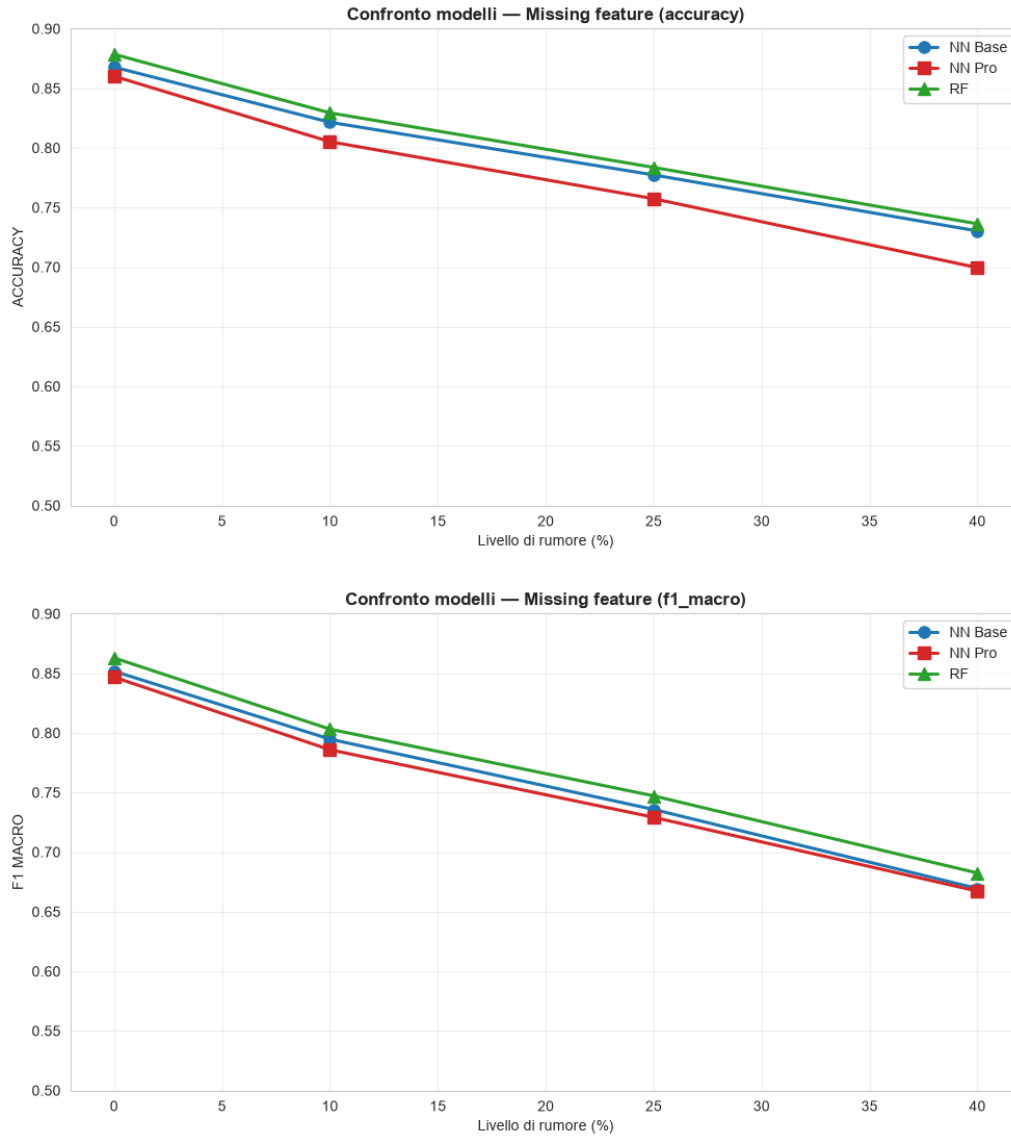


Figura 23: Missing feature: accuratezza (in alto) e F1 macro (in basso) rimuovendo le feature più importanti. Il calo è marcato fin dal primo livello, quando si elimina la sola **fAlpha**.

Questo risultato conferma in modo netto l'importanza di **fAlpha** emersa sia nei box-pot (Figura 3) sia nella feature importance (Figura 7): la sua rimozione, da sola, costa più del 40% di feature noise o di outlier. A differenza degli altri disturbi, qui non c'è rumore da filtrare ma informazione che semplicemente non esiste più; e poiché **fAlpha** è quasi scorrelata dalle altre feature (Figura 4), la sua perdita non può essere compensata, a differenza di quanto accade quando si disturba una feature ridondante. Notiamo che in questo scenario la rete Pro è la meno robusta (0,6995 al 40%): la regolarizzazione, utile contro il rumore, non offre alcun vantaggio quando il problema è la mancanza di segnale, e anzi la sua maggiore prudenza la penalizza leggermente. Le curve e la matrice di confusione al 40% (Figura 24) mostrano un modello che, privato delle feature più informative, fatica a separare le classi pur senza alcun segno di overfitting.

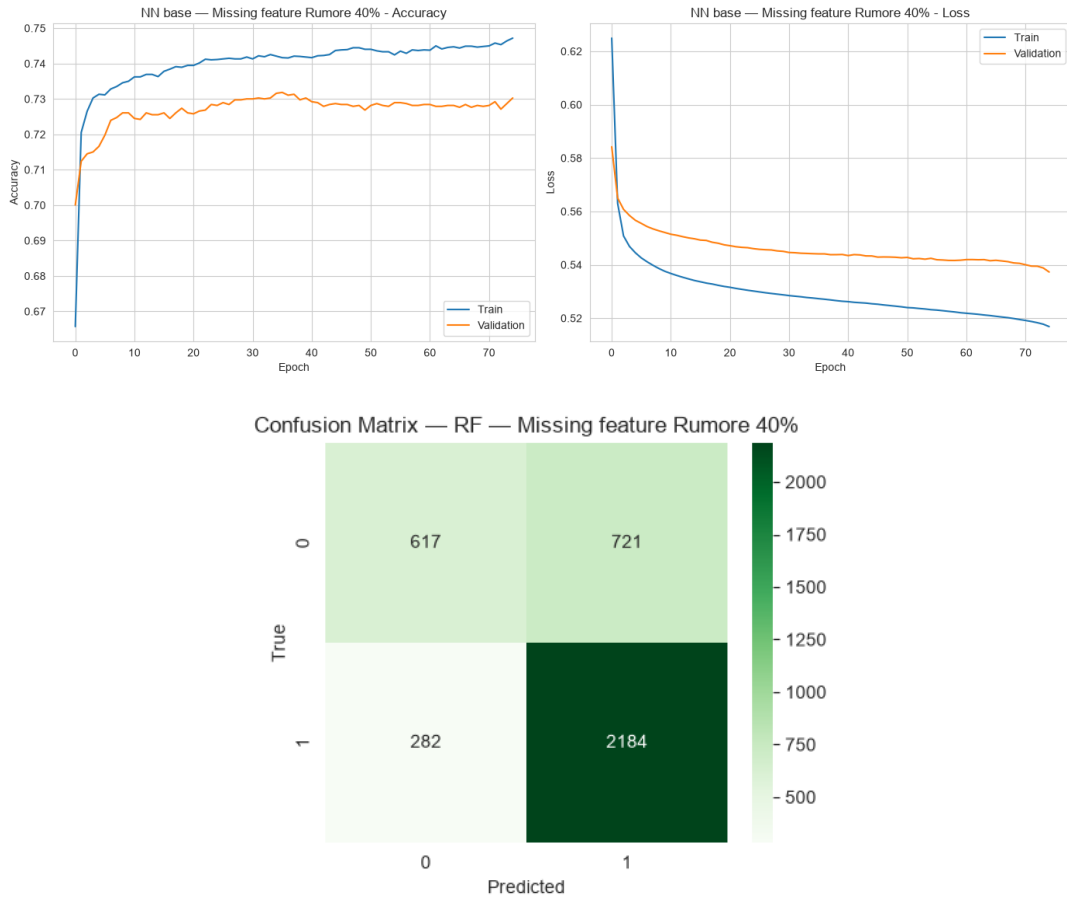


Figura 24: Missing feature al 40%: curve della rete Base (in alto) e matrice di confusione del Random Forest (in basso). Senza le feature più informative i modelli perdono potere discriminante, ma senza overfitting (curve train/validation vicine).

È in questo scenario che il degrado asimmetrico tra le classi raggiunge il suo massimo. Rimuovere le feature più informative cancella soprattutto la capacità di identificare gli adroni: al 40% la recall della classe minoritaria precipita a 0,43 per la rete Base e a 0,46 per il Random Forest, mentre i gamma restano sopra 0,89; persino la rete Pro, qui la meno accurata, conserva il vantaggio sugli adroni (0,55) grazie al riequilibrio delle classi. La progressione delle matrici di confusione della rete Base (Figura 25) mostra il deterioramento: man mano che spariscono le feature, una quota crescente di adroni viene etichettata come segnale, fino a comprometterne gravemente il riconoscimento.

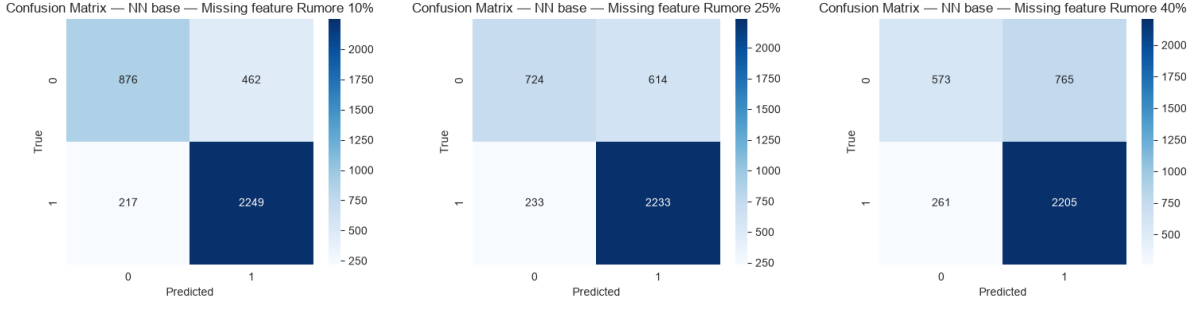


Figura 25: Missing feature, rete Base: matrici di confusione al 10%, 25% e 40% (da sinistra a destra). Rimuovendo le feature più informative il riconoscimento degli adroni (riga 0) si deteriora rapidamente.

10 Sintesi Trasversale

Avendo analizzato i sei rumori singolarmente, li mettiamo ora a confronto per leggere il quadro complessivo. La Figura 26 riassume, per ciascun modello e su un'unica scala, la robustezza a tutti i sei disturbi, sia in termini di accuratezza sia di F1 macro.

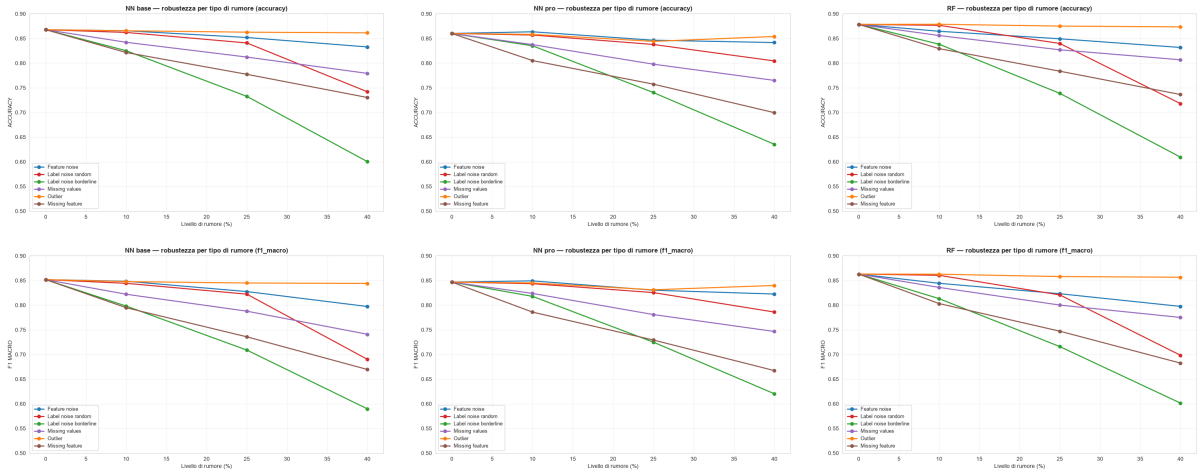


Figura 26: Robustezza per tipo di rumore: accuratezza (riga superiore) e F1 macro (riga inferiore) per rete Base, rete Pro e Random Forest (colonne da sinistra a destra). In tutti spiccano l'outlier come disturbo quasi innocuo e il label noise borderline come il più dannoso.

Da questa vista emerge con chiarezza una gerarchia di pericolosità dei rumori, comune a tutti i modelli. Gli outlier (curva in alto, quasi piatta) sono il disturbo più innocuo; seguono il feature noise e i valori mancanti, con cali moderati; poi il label noise random e il missing feature, più severi; e in fondo il label noise borderline, che provoca il crollo più ripido. La heatmap dell'accuratezza nel caso peggiore (Figura 27) condensa questa lettura in un'unica immagine: la colonna del borderline è la più “rossa” per tutti i modelli, quella degli outlier la più “verde”.

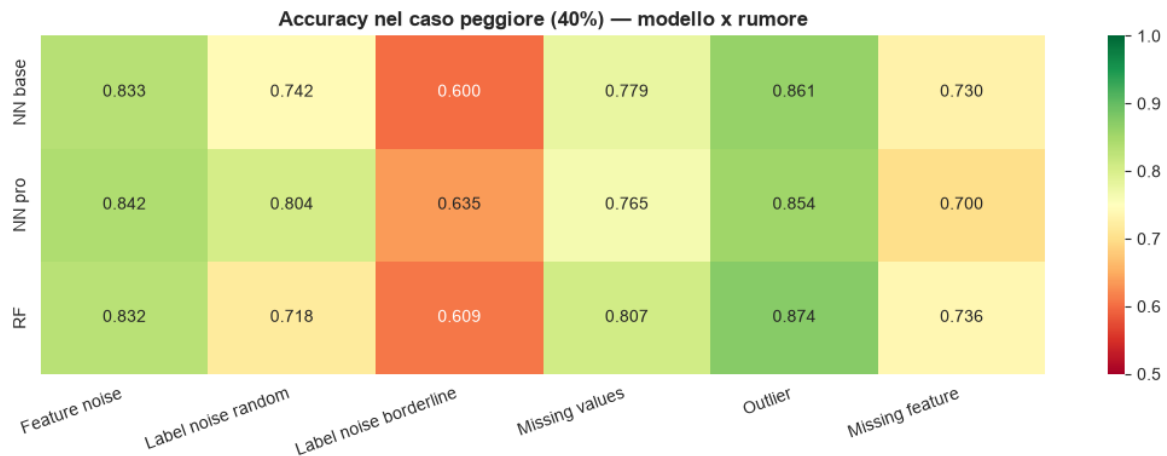


Figura 27: Accuratezza nel caso peggiore (40%) per ogni combinazione modello/rumore. Verde = prestazioni alte, rosso = basse. Il label noise borderline è sistematicamente il più dannoso.

Il grafico della degradazione (Figura 28) mostra invece di quanto cala ciascun modello passando dai dati puliti al 40% di rumore, e permette di confrontare i modelli sul piano della *stabilità* anziché della prestazione assoluta. Qui si apprezza il vantaggio della rete Pro sul rumore delle etichette casuali, dove subisce un calo molto minore degli altri due, e per contro la sua maggiore fragilità sul missing feature.

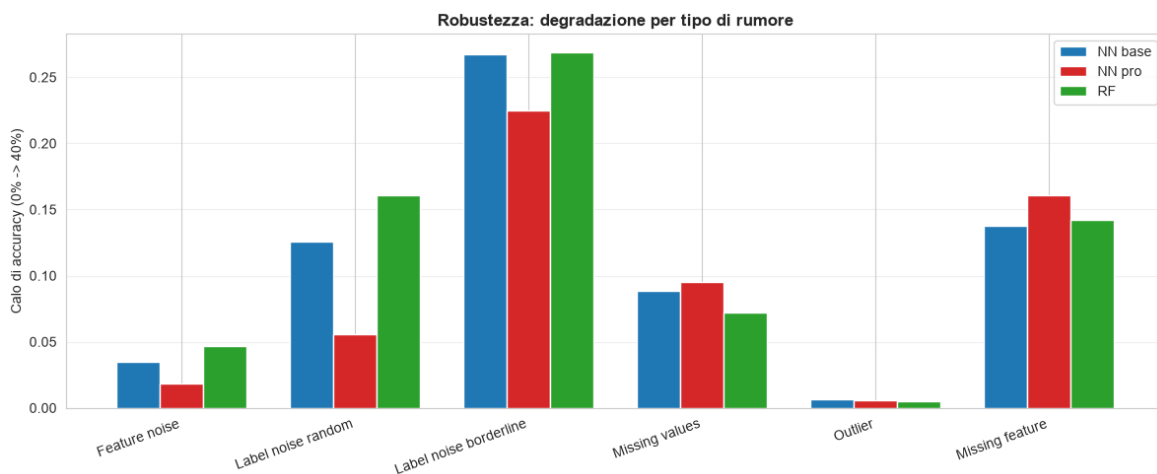


Figura 28: Calo di accuratezza (da 0% a 40%) per modello e tipo di rumore. Barre più basse indicano maggiore robustezza.

Sintesi del confronto

La gerarchia di pericolosità dei rumori è comune a tutti i modelli: outlier < feature noise $\approx$ missing values < label noise random $\approx$ missing feature < label noise borderline. Sul piano dei modelli non c'è un vincitore assoluto: la rete Pro guida sul rumore delle etichette, il Random Forest su valori mancanti e outlier, mentre il missing feature penalizza tutti perché elimina informazione insostituibile.

Riassumendo la robustezza complessiva con l'accuratezza media nel caso peggiore sui sei rumori (Figura 29), i tre modelli risultano molto vicini: la rete Pro guida di poco

(circa 0,767), seguita dal Random Forest (circa 0,763) e dalla rete Base (circa 0,758). È un esito che invita alla prudenza più che a proclamare un vincitore: la rete Pro vince grazie alla sua superiore tenuta sul rumore delle etichette e delle feature, ma paga sul missing feature; il Random Forest è il più solido su valori mancanti e outlier ma il più fragile sul label noise random. La scelta del modello, in altre parole, dovrebbe dipendere dal tipo di sporcizia che ci si attende nei dati reali.

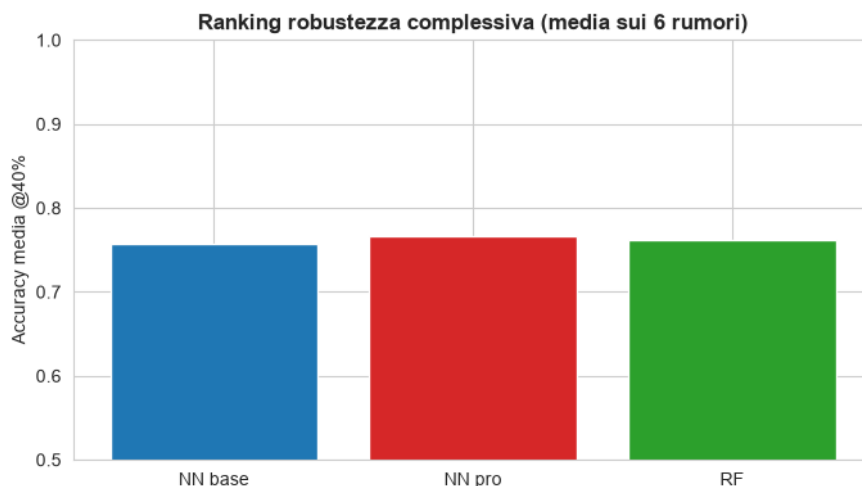


Figura 29: Robustezza complessiva: accuratezza media nel caso peggiore (40%) sui sei rumori. I tre modelli sono vicini, con un leggero vantaggio della rete Pro.

Infine, il confronto delle curve ROC tra dati puliti e 40% di feature noise (Figura 30) conferma sul piano del ranking quanto visto sull'accuratezza: le curve tratteggiate (rumore) restano vicine a quelle continue (pulito), a riprova che il feature noise erode solo marginalmente la capacità dei modelli di ordinare correttamente i campioni. Lo scostamento più visibile riguarda la rete Base, coerentemente con la sua maggiore tendenza ad adattarsi al rumore osservata nella Sezione 9.

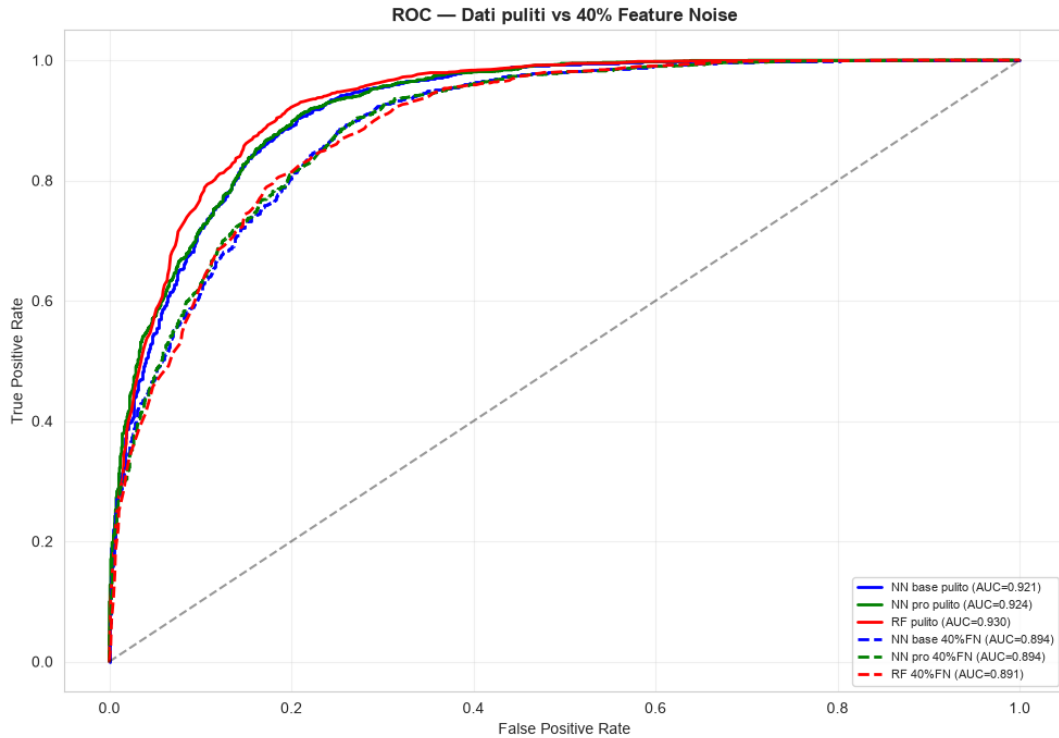


Figura 30: Curve ROC dei tre modelli su dati puliti (continue) e con 40% di feature noise (tratteggiate). Lo scostamento è contenuto: il feature noise incide poco sul potere di ranking.

11 Analisi Non Supervisionata (K-Means)

Per completare lo studio abbiamo aggiunto un esperimento non supervisionato con l'algoritmo K-Means (con $k = 2$), che raggruppa i dati usando solo le feature, senza mai vedere le etichette. Questo ci permette di osservare un contrasto interessante: il rumore sulle etichette, che abbiamo visto essere il più dannoso per i modelli supervisionati (Sezione 9), non ha per costruzione *alcun* effetto sul K-Means, mentre il rumore sulle feature ne degrada la qualità.

La prima osservazione, però, è che i cluster geometrici trovati dal K-Means non corrispondono alle classi fisiche. Sui dati puliti otteniamo un indice ARI di appena 0,006 e una purezza di 0,648: la purezza coincide quasi esattamente con la proporzione della classe maggioritaria stimata nell'analisi esplorativa (Sezione 4), il che significa che il clustering non recupera la distinzione gamma/adrone meglio di quanto farebbe assegnando tutto alla classe più frequente. La Figura 31, che proietta i dati sulle prime due componenti principali, spiega visivamente il perché: i due cluster trovati dal K-Means dividono lo spazio lungo la direzione di massima varianza, ma le classi reali sono ampiamente sovrapposte e non separabili da una semplice partizione geometrica. In altre parole, la struttura che separa gamma e adroni non coincide con la struttura di varianza dominante del dataset, e un algoritmo non supervisionato basato sulle distanze non riesce a coglierla.

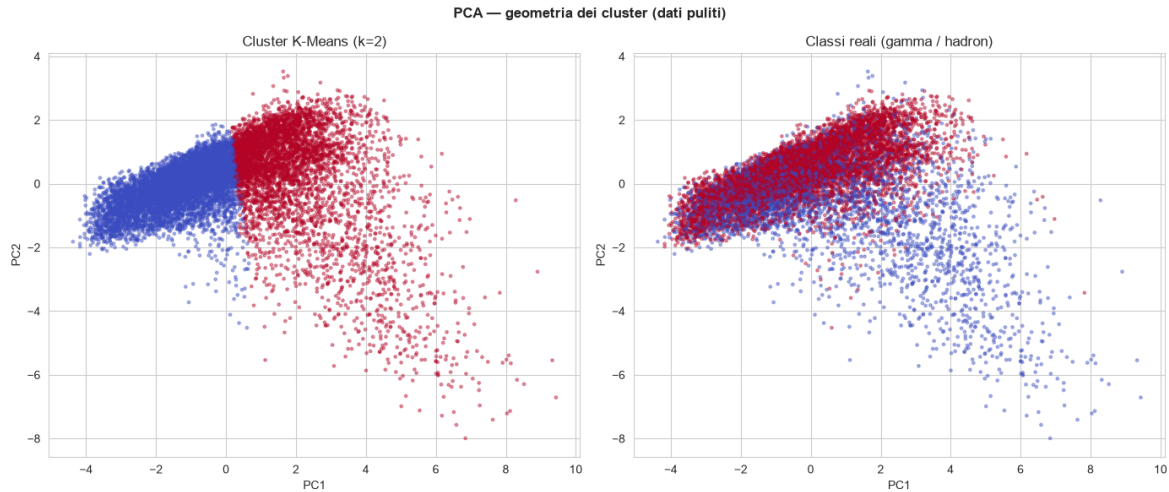


Figura 31: Proiezione PCA dei dati: a sinistra i due cluster trovati dal K-Means, a destra le classi reali. I cluster geometrici non ricalcano la separazione fisica gamma/adrone ($ARI \approx 0$).

Riguardo al rumore, i risultati confermano le nostre attese. Aumentando il feature noise la qualità geometrica dei cluster, misurata dalla silhouette, si degrada progressivamente (da 0,290 sui dati puliti fino a 0,102 con deviazione standard 2,0), perché il rumore sfuma i confini tra i raggruppamenti. Il label noise, al contrario, lascia i cluster esattamente invariati: confrontando il clustering ottenuto sui dati puliti con quello ottenuto dopo aver invertito il 40% delle etichette otteniamo un ARI pari a 1,000, cioè partizioni identiche. Questo perché il K-Means non usa mai le etichette, e quindi corromperle non può spostare i centroidi. È una conferma quasi tautologica ma utile, perché evidenzia una differenza concettuale importante: la vulnerabilità di un modello a un certo tipo di rumore dipende da *quali* informazioni quel modello effettivamente utilizza.

12 Conclusioni

Questo studio ci ha permesso di trasformare la domanda generica “quale modello è migliore” in una domanda più precisa e più utile: “quale modello regge meglio quale tipo di sporcizia”. I risultati, tutti ricavati direttamente dall’esecuzione del notebook, disegnano un quadro coerente e ricco di compromessi.

Sul piano dei rumori abbiamo trovato una gerarchia netta di pericolosità. Gli **outlier** sono il disturbo più innocuo: con un minimo di trattamento, o persino senza, l’impatto è trascurabile. Il **feature noise** e i **valori mancanti** provocano cali moderati e gestibili, complici la ridondanza tra le feature e l’efficacia dell’imputazione. Il **missing feature** e il **label noise** sono invece i disturbi gravi, ma per ragioni opposte: il primo perché elimina informazione insostituibile (la rimozione della sola **fAlpha** costa più del massimo livello di feature noise), il secondo perché induce le reti a memorizzare gli errori. Il caso peggiore in assoluto è il **label noise borderline**, che corrompendo i campioni vicini al confine decisionale spinge i modelli verso prestazioni prossime al classificatore banale.

Sul piano dei modelli, nessuno domina su tutti i fronti. La **rete Pro** ha confermato il valore della regolarizzazione: l’early stopping, in particolare, la rende molto più resistente al rumore sulle etichette casuali, dove impedisce la memorizzazione che affonda la rete Base. Tuttavia la stessa prudenza non aiuta, e anzi penalizza leggermente, quando il

problema è la mancanza di feature. Il **Random Forest** è il più solido su dati puliti, valori mancanti e outlier, ma il più fragile proprio sul label noise random. La **rete Base**, infine, è competitiva sui dati puliti ma la più esposta non appena il rumore aumenta. La conseguenza pratica è che la scelta del modello non andrebbe fatta in astratto, ma in funzione del tipo di errore che ci si aspetta di incontrare nei dati reali.

La lettura per classe, che abbiamo intrecciato all'analisi dei singoli rumori, ha aggiunto a questo quadro una sfumatura importante: sotto la maggior parte dei rumori a degradarsi non sono le due classi in egual misura, ma quasi solo quella minoritaria degli adroni, mentre i gamma continuano a essere riconosciuti. L'accuratezza, da sola, avrebbe mascherato questo collasso silenzioso; è anche per questo che il riequilibrio delle classi della rete Pro, pur non migliorando di molto l'accuratezza, si rivela prezioso quando interessa riconoscere bene la classe rara. Fa eccezione il solo label noise borderline, che degrada entrambe le classi in egual misura perché ne falsifica il confine anziché la proporzione.

Crediamo che la lezione più importante riguardi però il preprocessing più che il modello. I nostri esperimenti mostrano che strategie semplici e mirate — imputare con la mediana piuttosto che con la media, winsorizzare gli outlier, monitorare la perdita di validation per fermare l'addestramento — recuperano gran parte delle prestazioni perse, mentre nessun modello, per quanto sofisticato, può recuperare un'informazione che è stata cancellata (feature mancanti) o falsificata nel punto più sensibile (label borderline). La robustezza, in definitiva, si costruisce tanto a monte, nella qualità e nel trattamento dei dati, quanto a valle nella scelta dell'algoritmo. L'esperimento non supervisionato ci ha infine ricordato che la vulnerabilità a un rumore dipende da quali informazioni il modello usa: ciò che è devastante per un classificatore supervisionato può essere del tutto irrilevante per un metodo che quelle informazioni non le guarda affatto.